

STM32 NUCLEO - INTERFAZ DE CONTROL DE LED

Rodrigo Castillejos

TABLA DE CONTENIDOS

1	Configuración del espacio de trabajo	2
1.1	Requisitos Previos del Sistema	2
1.2	Verificación de .NET SDK 9	2
1.3	Verificación de Node.js	3
1.3.1	Comprobar la versión de Node.js	3
1.3.2	Comprobar la versión de npm	3
1.4	Verificación de Git	3
2	Estructura de carpetas y creación del proyecto	5
3	Inicialización del backend (C# .NET 9 Web API)	7
3.1	Desde la terminal	7
3.2	Crear el proyecto Web API de forma visual	9
3.2.1	Instalar el paquete System.IO.Ports	11
4	Inicialización del Frontend	13
5	Gestión de espacios de trabajo (workspaces)	15
5.1	El archivo Workspace de VS Code	15
5.2	Crear archivo .gitignore	16
6	Compilación y ejecución del proyecto	18
6.1	Compilación y ejecución del backend	18
6.2	Compilación y ejecución del frontend	18
7	title	19
7.1	Crear el Servicio Serial (SerialService)	19
7.1.1	Namespaces	20
7.1.2	Declaración de variables	21
7.1.3	Constructor de la clase	23
7.1.4	Contenido de la clase	25
7.1.5	Código completo	38
7.2	Crear el Hub de SignalR (SerialHub.cs)	43
7.2.1	¿Qué significan los componentes de este código?	44
7.3	Crear el controlador API (SerialController.cs)	45
7.4	Configuración del archivo Program.cs	47

1 CONFIGURACIÓN DEL ESPACIO DE TRABAJO

1.1 Requisitos Previos del Sistema

Antes de abrir VS Code, hay que asegurarse de tener instalado lo siguiente:

.NET SDK 9: Se puede descargar del sitio oficial de Microsoft. Es necesario para compilar y ejecutar el backend en C#.

NODE.JS (VERSION LTS RECOMENDADA): Se puede descargar del sitio oficial de Node.js

GIT: Se descarga Git para Windows para el control de versiones.

1.2 Verificación de .NET SDK 9

Para verificar si se tiene instalado **.NET SDK 9** en el sistema, abre la terminal (ya sea en PowerShell, CMD, o la terminal integrada de VS Code) y ejecuta uno de los siguientes comandos:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>dotnet --version
```

El sistema debe devolver un número de versión, por ejemplo:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>dotnet --version
9.0.100
```

Si se tiene múltiples versiones instaladas, se puede ejecutar el siguiente comando:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>dotnet --list-sdks
```

El sistema deberá devolver todas las versiones instaladas junto a su ubicación en el disco, ejemplo:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>dotnet --list-sdks
8.0.204 [C:\Program Files\dotnet\sdk]
9.0.100 [C:\Program Files\dotnet\sdk]
```

Si al ejecutar el comando la terminal devuelve el siguiente mensaje, significa que no tienes el SDK instalado o no se ha agregado el PATH al sistema. En este caso, puedes descargarlo directamente desde dot.net seleccionando la opción **.NET SDK** (no el "Runtime").

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>dotnet --version
The command could not be loaded, possibly because:
    * You intended to execute a .NET application:
      The application '--version' does not exist.
    * You intended to execute a .NET SDK command:
      No -NET SDKs were found.

Download a .NET SDK:
https://aka.ms/dotnet/download

Learn about SDK resolution:
https://aka.ms/dotnet/sdk-not-found

C:\Users\casti>
```

1.3 Verificación de Node.js

1.3.1 Comprobar la versión de Node.js

Para saber si se tiene instalado Node.js (y su gestor de paquetes **npm** que es indispensable para Angular), abre tu terminal y ejecuta los siguientes comandos:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>node -v
```

El sistema deberá devolver un número de versión de npm, por ejemplo:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>node -v
v22.12.0
```

Si al ejecutar el comando la terminal devuelve el siguiente mensaje significa que no tienes Node.js instalado en tu sistema. Puedes descargar el instalador desde la página oficial de nodejs.org

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>node -v
'node' is not recognized as an internal or external command,
operable program or batch file.
```

1.3.2 Comprobar la versión de npm

Node.js se instala junto con **npm**. Es buena idea comprobar que también responda correctamente:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>npm -v
10.9.0
```

1.4 Verificación de Git

Para comprobar si tiene **Git** instalado en su computadora, abra su terminal y ejecute el siguiente comando:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>git --version
10.9.0
```

Si Git está instalado, el sistema deberá devolver la versión Git configurada en tu sistema, por ejemplo:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>git --version
git version 2.52.0.windows.1
```

Si quiere verificar en dónde se encuentra el ejecutable puede ejecutar el siguiente comando

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\casti>where git
C:\Program Files\Git\cmd\git.exe
```

Si la terminal muestra el siguiente mensaje significa que no tiene instalado Git en su computadora. Puede descargar el instalador oficial para Windows desde git-scm.com

```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. Todos los derechos reservados.  
  
C:\Users\casti>git --version  
'git' is not recognized as an internal or external command,  
operable program or batch file.
```

Ejecución del instalador de Git

Durante los pasos de instalación, asegúrate de dejar activada la opción predeterminada que dice **"Git from the command line and also from 3rd-party software"** (esto garantiza que puede usar git desde VS Code y PowerShell).

2 ESTRUCTURA DE CARPETAS Y CREACIÓN DEL PROYECTO

Se crea una carpeta en `C:\Visual Studio Projects`. Nos posicionamos en esta ruta y abrimos una nueva terminal, tal como se muestra en la figura 1.

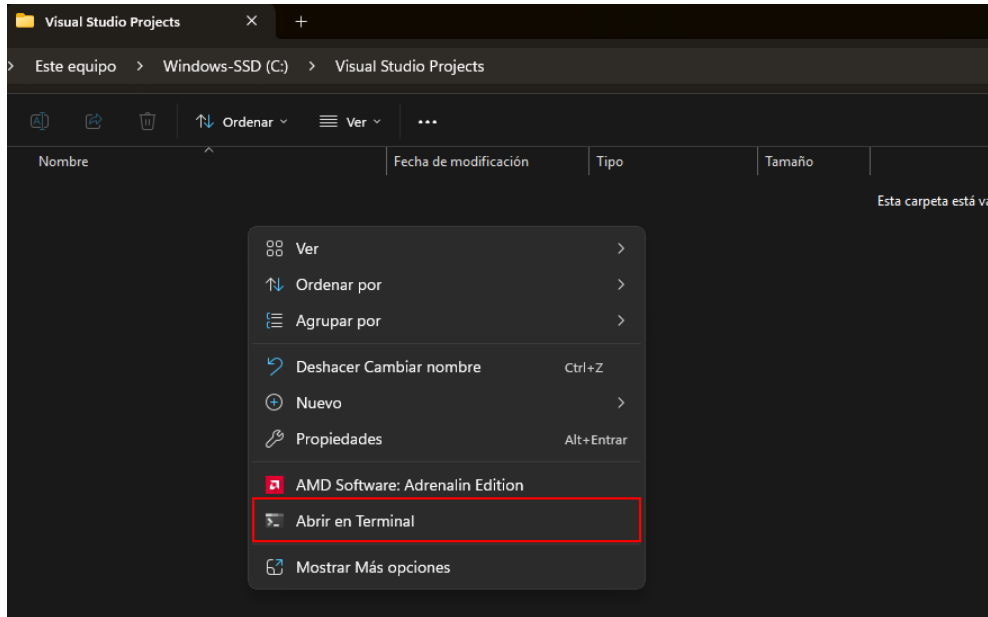


Figura 1: Abrir una nueva terminal en `C:\Visual Studio Projects`

Ejecutamos los siguientes comandos para crear la estructura base del proyecto.

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Visual Studio Projects>mkdir NucleoBoard_LedControl
C:\Visual Studio Projects>cd NucleoBoard_LedControl
```

Una vez creada la carpeta principal, crearemos el contenedor de la solución. Esto define el punto de partida de todo el proyecto:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Visual Studio Projects\NucleoBoard_LedControl>dotnet new sln -n NucleoBoard_LedControl
```

El comando de .NET CLI `dotnet new sln -n NucleoBoard_LedControl` sirve para crear una nueva solución (.sln) en .NET con el nombre **NucleoBoard_LedControl**.

Ejecución del instalador

1. **dotnet**: Es la herramienta de línea de comandos de .NET. Con ella puedes crear proyectos, compilar, ejecutar, restaurar paquetes, publicar, etc.
2. **new**: Indica que quieres crear algo nuevo a partir de una plantilla.
3. **sln**: Es la plantilla para crear una solución de Visual Studio / .NET. Una solución (.sln) es como un contenedor organizador donde puedes agrupar varios proyectos relacionados.
4. **-n NucleoBoard_LedControl**: La opción **-n** significa **name** (nombre).

Creamos los subdirectorios dentro de `NucleoBoard_LedControl` para separar el servidor del cliente.

```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. Todos los derechos reservados.  
  
C:\Visual Studio Projects\NucleoBoard_LedControl>mkdir Service  
C:\Visual Studio Projects\NucleoBoard_LedControl>mkdir View
```

Estos comandos permiten crear las carpetas que se muestran en la figura 2.

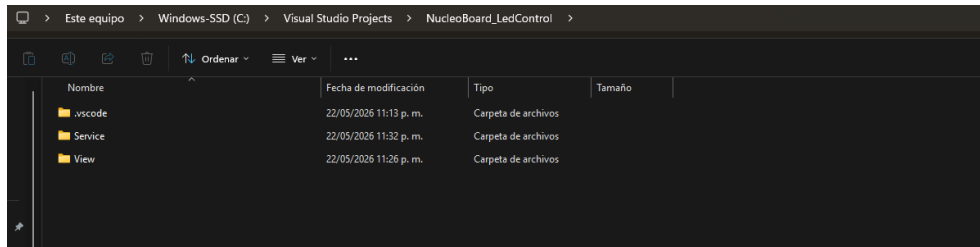


Figura 2: Creación de las carpetas del proyecto.

3 INICIALIZACIÓN DEL BACKEND (C# .NET 9 WEB API)

Desde la terminal raíz del proyecto, inicializa la API Web de C# y agrega el driver del puerto serie.

3.1 Desde la terminal

Desde la terminal raíz del proyecto `C:\Visual Studio Projects\NucleoBoard_LedControl` inicializa la API web de C# y agrega el driver del puerto serie utilizando los siguientes comandos:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

# 1. Muevete a la carpeta Service
C:\Visual Studio Projects\NucleoBoard_LedControl>cd Service

#2. Crea un nuevo proyecto Web API de C# .NET 9
C:\Visual Studio Projects\NucleoBoard_LedControl\Service>dotnet new webapi --use-controllers -o
LedControllerService

#3. Regresamos a la raíz
C:\Visual Studio Projects\NucleoBoard_LedControl\Service>cd ..
```

El comando de .NET CLI `dotnet new webapi -use-controllers -o LedControllerService` crea un nuevo proyecto de **ASP.NET Core Web API**, el comando se explica a continuación:

1. **dotnet**: Es la herramienta de línea de comandos de .NET. Se usa para crear proyectos, compilarlos, ejecutarlos, restarurar paquetes, publicar, etc.
2. **new**: Indica que quieres crear algo nuevo. En este caso, crearás un proyecto basado en una plantilla.
3. **webapi**: Es la plantilla que se va a usar. La plantilla **webapi** genera un proyecto de **API REST** con **ASP.NET Core**.
4. **--use-controllers**: Esta opción le dice a la plantilla que use el enfoque de **controladores MVC** en lugar de depender solo de **Minimal APIs**. Con esta opción, el proyecto estará preparado para trabajar con clases como estas:

```
1 [ApiController]
2 [Route("api/[controller]")]
3 public class LedController : ControllerBase
4 {
5     [HttpGet]
6     public IActionResult Get()
7     {
8         return Ok("LED encendido");
9     }
10 }
```

5. **-o LedControllerService**: La opción **-o** significa **output**. Le indica a dotnet que cree el proyecto dentro de una carpeta llamada `LedControllerService`. Si la carpeta no existe, la crea automáticamente.

Justo después de crear el proyecto, lo vinculamos a la solución para que el IDE sepa que forman parte del mismo ecosistema:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Visual Studio Projects\NucleoBoard_LedControl>dotnet sln add Service/LedControllerService/
LedControllerService.csproj
```

Una vez que el proyecto está creado y vinculado, entramos a su carpeta para agregar las dependencias de terceros (como el puerto serie).

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Visual Studio Projects\NucleoBoard_LedControl>cd Service
C:\Visual Studio Projects\NucleoBoard_LedControl\Service>cd LedControllerService
C:\Visual Studio Projects\NucleoBoard_LedControl\Service\LedControllerService>dotnet add
package System.IO.Ports --version 10.0.8
```

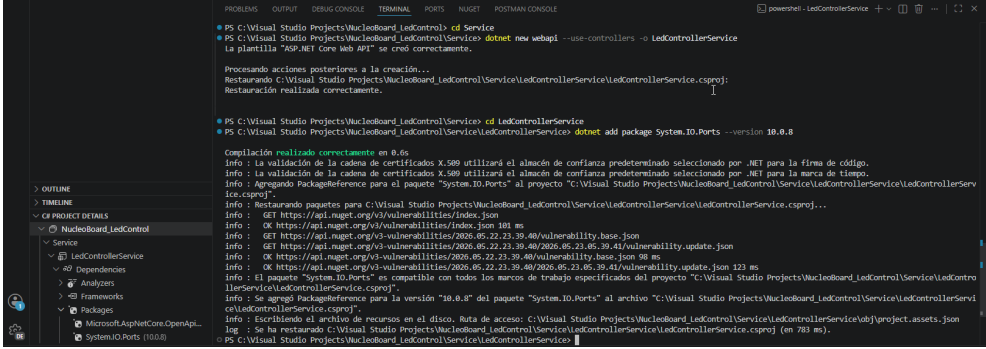


Figura 3: Crear nueva plantilla e instalar paquete Nuget.

El comando `dotnet add package System.IO.Ports -version 10.0.8` sirve para agregar una dependencia NuGet a tu proyecto .NET, específicamente el paquete `System.IO.Ports` en la versión `10.0.8`

1. **dotnet**: Es la herramienta de línea de comandos de .NET CLI, usada para crear, compilar, ejecutar y administrar proyectos y paquetes.
2. **add package**: Indica que quieres agregar un paquete NuGet al proyecto actual.
3. **System.IO.Ports**: Es el nombre del paquete NuGet que proporciona clases para trabajar con puertos seriales; el tipo principal que expone es `SerialPort`.
4. **--version 10.0.8**: Le indica a la CLI que agregue exactamente esa versión del paquete, en lugar de usar automáticamente la última versión compatible disponible.

Cuando ejecutas ese comando, .NET hace varias cosas automáticamente:

Primero verifica la compatibilidad entre el paquete y los frameworks del proyecto; si es compatible, agrega (o actualiza) un elemento `<PackageReference>` en el archivo `.csproj`; y después ejecuta **dotnet restore** para descargar el paquete y generar los archivos de assets necesarios.

En la práctica, tu archivo `.csproj` terminaría con algo parecido a esto

```
<ItemGroup>
  <PackageReference Include="System.IO.Ports" Version="10.0.8" />
</ItemGroup>
```

Sin -version

La herramienta intentará resolver una versión apropiada del paquete, normalmente la más reciente compatible con tu proyecto. La documentación indica que, si el paquete ya existe, el comando también puede actualizar la referencia a una versión compatible.

3.2 Crear el proyecto Web API de forma visual

Si tienes instalada la extensión **C# Dev Kit** (figura 4), puedes crear proyectos utilizando el asistente gráfico incorporado.

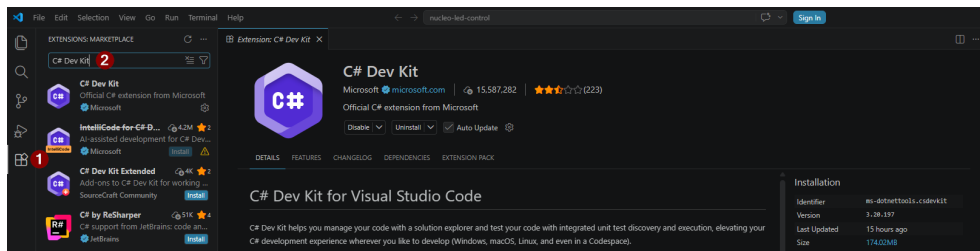


Figura 4: Instalación de la extensión C# Dev Kit

Abre VS Code y abre la carpeta vacía `nucleo-led-control` (figura 5)

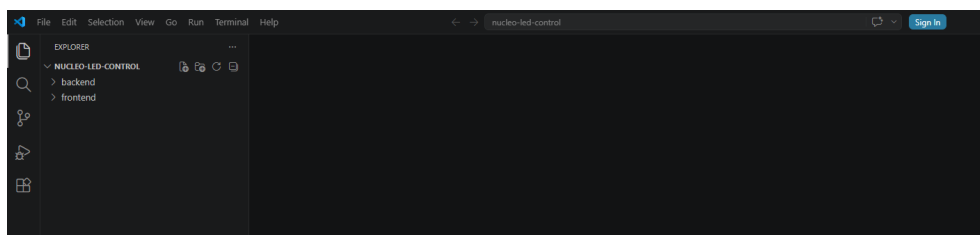


Figura 5: Abriendo la carpeta `nucleo-led-control` en VS Code.

Abre la **Paleta de Comandos** presionando `Ctrl` + `Shift` + `P` o `F1` (figura 6).

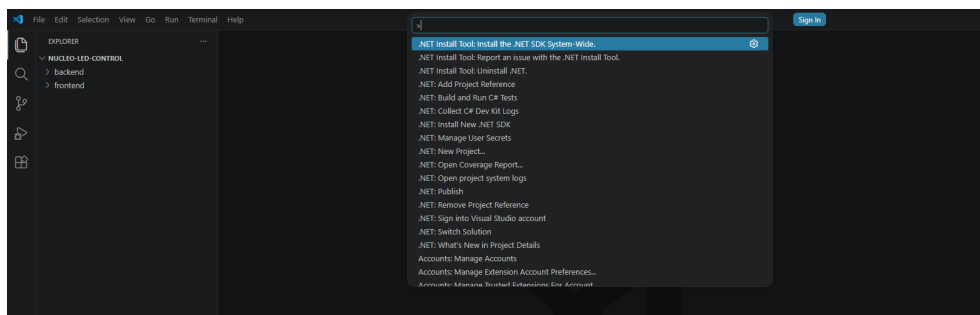


Figura 6: Abriendo la Paleta de Comandos en VS Code.

Escriba el comando `.NET: New Project...` (Nuevo proyecto .NET) y presiona Enter (figura 7).

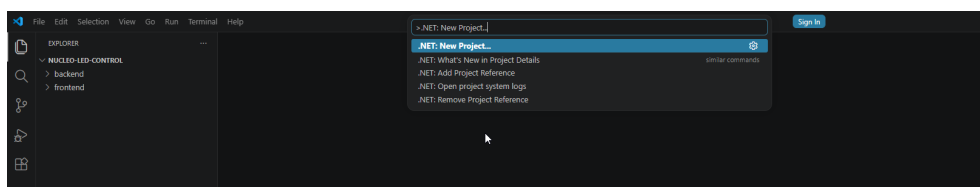


Figura 7: Nuevo proyecto .NET.

El asistente mostrará una lista visual de plantillas. Selecciona **ASP.NET Core Web API** (figura 8).

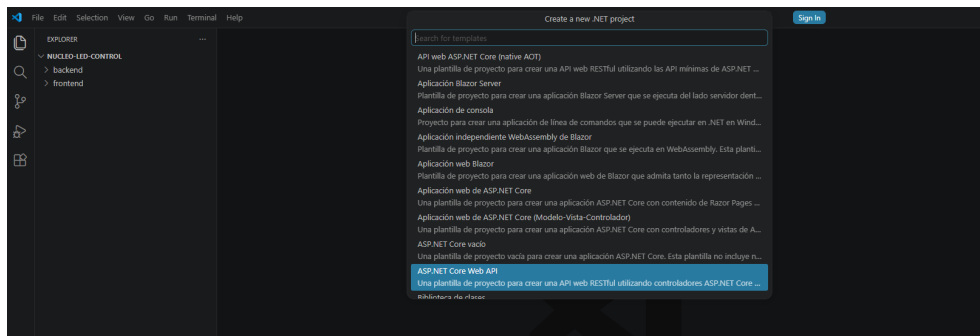


Figura 8: Lista de plantillas.

El asistente te pedirá tres cosas de forma guiada:

- **Nombre del proyecto:** Escribe `backend`

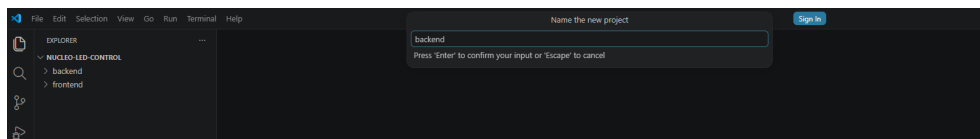


Figura 9: Solicitud del nombre del proyecto.

- **Directorio de salida:** Selecciona la carpeta donde quieres crearlo.

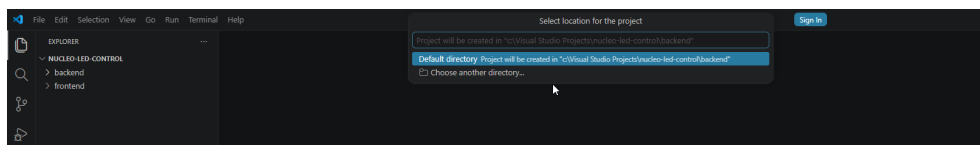


Figura 10: Directorio de salida.

- **Configuraciones adicionales:** (Como si quieres usar controladores o Minimal APIs, a través de menús desplegables simples).

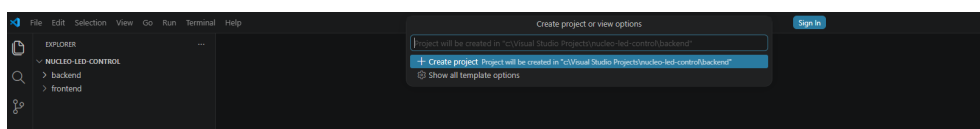


Figura 11: Finalizar la creación del proyecto.

Al terminar, la extensión generará automáticamente todos los archivos del proyecto (figura 12).

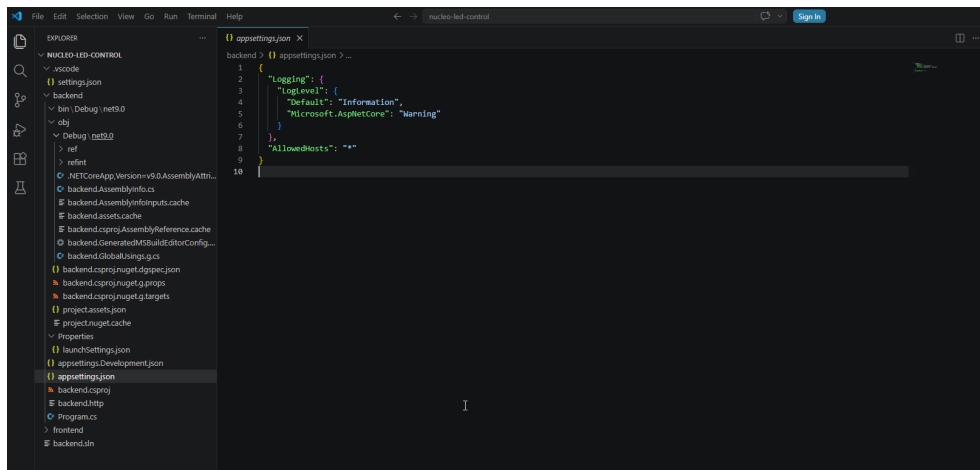


Figura 12: Finalizar la creación del proyecto.

3.2.1 Instalar el paquete System.IO.Ports

Para instalar librerías sin usar comandos, la mejor opción es instalar la extensión NuGet Gallery en VS Code (figura 13).

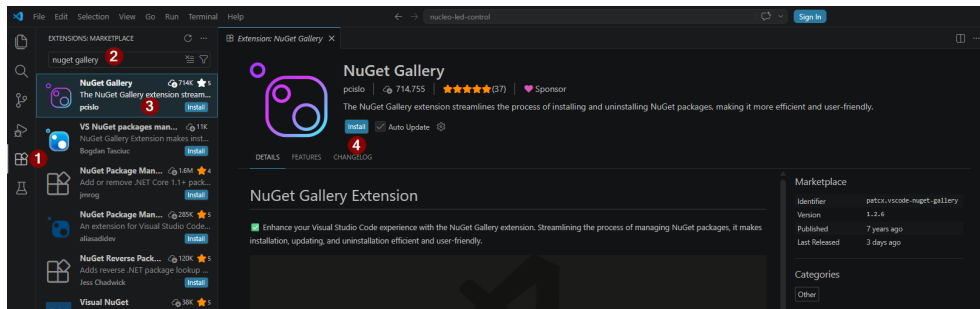


Figura 13: Instalación de la extensión NuGet Gallery.

Abre la **Paleta de Comandos**, busca y selecciona **Nuget: Focus on Nuget View** (figura 14).

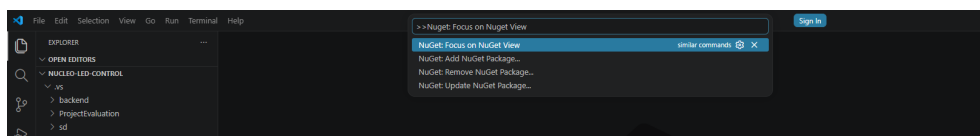


Figura 14: Abriendo la vista de paquetes NuGet.

Se abrirá una pestaña con una interfaz web integrada de búsqueda. En la barra de búsqueda escriba **System.IO.Ports**. Selecciona el paquete en la lista de resultados y selecciona el ícono que contiene el símbolo de "+" para instalar el paquete.

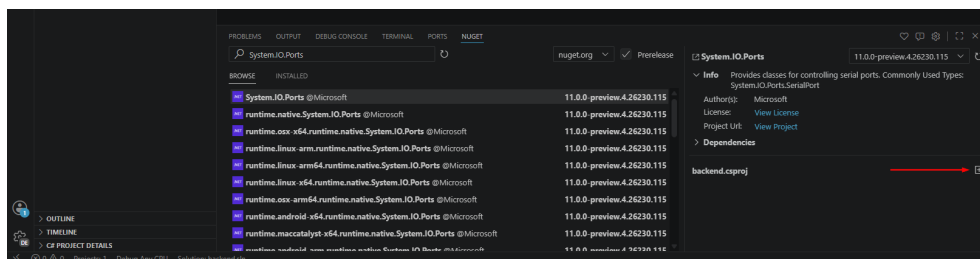


Figura 15: Instalación del paquete System.IO.Ports

Para ver los paquetes NuGet instalados así como las dependencias virtuales, busca el panel llamado **C# Project Details**. Si está colapsado, has clic en la flecha a la izquierda del título para desplegarlo (figura 16). Allí podrás ver de forma virtual las dependencias NuGet, paquetes instalados, etc.

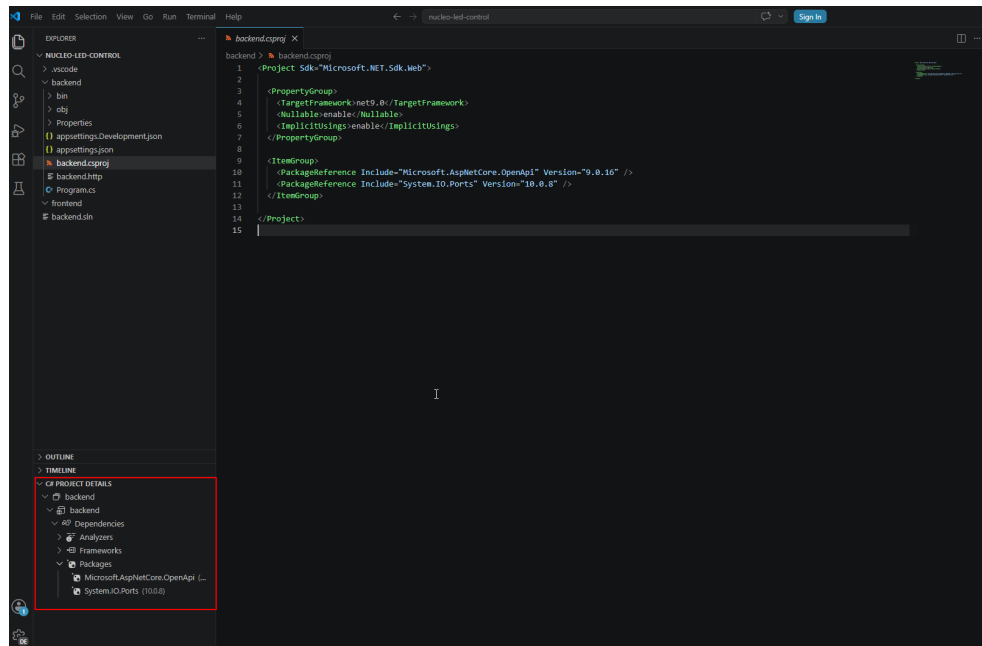


Figura 16: Panel C# Project Details para ver los paquetes instalados.

4 INICIALIZACIÓN DEL FRONTEND

Ejecute el siguiente comando (en una nueva terminal de VS Code, por ejemplo)

```
npx -y @angular/cli new frontend -directory=frontend -style=css -routing=true
-ssr=false
```

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.
```

```
CPS C:\Visual Studio Projects\nucleo-led-control\frontend>npx -y @angular/cli new frontend --
directory=frontend --style=css --routing=true --ssr=false
```

Este comando es una instrucción optimizada para crear un nuevo proyecto de Angular nuevo y listo para usar sin necesidad de configurar nada manualmente.

1. **npx** (Node Package Runner): Es una herramienta que viene con Node.js y sirve para ejecutar paquetes sin tener que instalarlos de forma global en tu computadora. En lugar de instalar permanentemente el CLI de Angular (`npm install -g @angular/cli`), **npx** descarga una versión temporal de la herramienta en la memoria, crea tu proyecto, y luego libera el espacio. Esto mantiene tu sistema limpio.
2. **-y** (Yes): Es un parámetro para omitir preguntas. Le dice a Node: "Si me vas a preguntar si deseo descargar el paquete de Angular CLI, responde 'Sí' automáticamente". Esto hace que el comando se ejecute sin pausas interactivas.
3. **@angular/cli**: Es la librería oficial de la línea de comandos de Angular (Angular CLI). Es la herramienta responsable de estructurar la aplicación, configurar el compilador, y generar los archivos base.
4. **new frontend**: Es el comando interno de Angular CLI para crear un nuevo proyecto. El nombre que se le asignará al proyecto y a la aplicación principal es **frontend**.
5. **--directory=frontend**: Le indica al instalador dónde crear los archivos. Por defecto, Angular crea una nueva carpeta con el nombre del proyecto. Con **--directory=frontend**, le indicamos que coloque todos los archivos directamente dentro de la carpeta frontend, evitando que se genera una estructura redundante `frontend/frontend/...`
6. **--style=css**: Define qué motor de estilos utilizará la aplicación. En este caso, le decimos que configure hojas de estilo en CSS tradicional (Vanilla CSS) en lugar de preprocesadores más complejos como SCSS, Sass o Less.
7. **--routing=true**: Configura automáticamente el sistema de rutas de Angular (Angular Router). Crea los archivos de configuración de navegación (como `app.route.ts`) permitiendo que puedas definir páginas y secciones de URL en el futuro muy fácilmente.
8. **--ssr=false**: Desactiva el **Server-Side Rendering (SSR)**. Angular moderno intenta activar SSR por defecto (lo que significa que un servidor Node procesa el HTML inicial). Como nosotros estamos creando una arquitectura limpia donde el servidor es de C# .NET, solo necesitamos que Angular compile como una aplicación de cliente clásica (Single Page Application - SPA), la cual es más rápida de compilar, más ligera y se ejecuta directamente en el navegador del usuario.

Al finalizar, la terminal deberá arrojar el siguiente que se muestra en la figura 17.

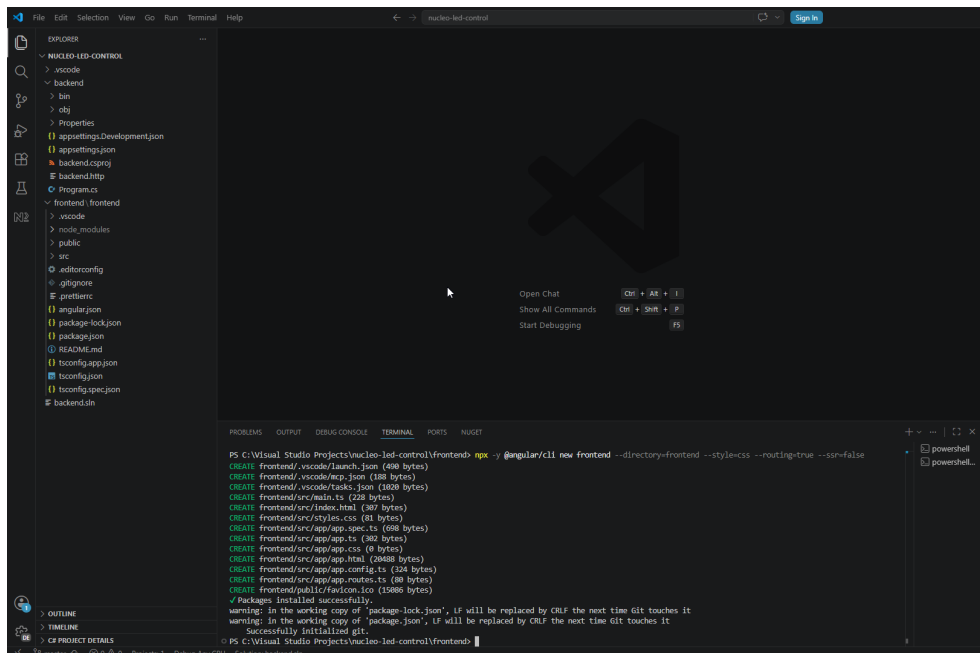


Figura 17: Instalación de paquetes.

Ahora ejecute el directorio a **frontend** y ejecute el siguiente comando (figura 18)
npm install @microsoft/signalr.

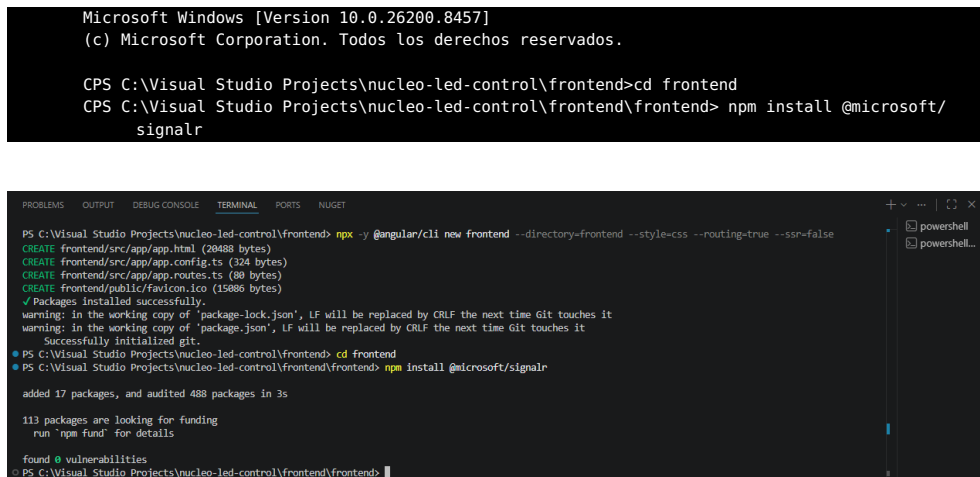


Figura 18: Instalación del SDK del cliente de SignalR para JavaScript/TypeScript.

Ejecución de comandos

Se pueden ejecutar estos comandos utilizando cualquier terminal estándar (CMD en Windows, PowerShell, Git Bash o la terminal integrada de VS Code). No es obligatorio utilizar específicamente *Node.js Command prompt*.

Cuando instalas Node.js, el instalador de Windows configura automáticamente las variables de entorno (PATH) de tu sistema. Esto permite que Windows reconozca comandos como **node**, **npm** y **npm** de forma global desde cualquier ventana de comandos.

5 GESTIÓN DE ESPACIOS DE TRABAJO (WORKSPACES)

5.1 El archivo Workspace de VS Code

Para no tener carpetas anidadas en el explorador y ver tus carpetas Service y View de forma limpia y organizada en la barra lateral, debes crear un archivo en la raíz llamado: `NucleoBoard_LedControl.code-workspace`

```

1  {
2      "folders": [
3      {
4          "name": "CONFIGURACION GENERAL",
5          "path": "C:/Visual Studio Projects/
        NucleoBoard_LedControl"
6      },
7      {
8          "name": "SERVICIO BACKEND (C#)",
9          "path": "C:/Visual Studio Projects/
        NucleoBoard_LedControl/Service/LedControllerService"
10     },
11     {
12         "name": "VISTA FRONTEND (Angular)",
13         "path": "C:/Visual Studio Projects/
        NucleoBoard_LedControl/View/LedControlView"
14     }
15 ],
16 "settings": {
17     "editor.formatOnSave": true,
18     "editor.defaultFormatter": "esbenp.prettier-vscode"
19 }
20 }
```

Cree el archivo en blanco a través del siguiente comando (figura 19):

```

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.
```

```
C:\Visual Studio Projects\NucleoBoard_LedControl>code NucleoBoard_LedControl.code-workspace
```

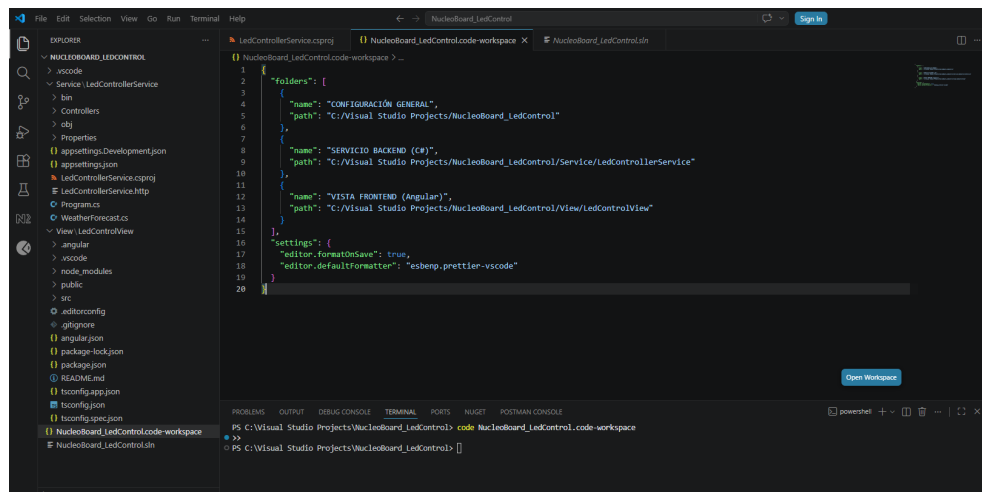


Figura 19: Archivo Workspace en la raíz del proyecto.

5.2 Crear archivo .gitignore

Actualmente, Angular tiene su propio .gitignore dentro de View/LedControlView/, pero no hay un archivo global en la raíz. Si deseas subir el proyecto a Git, se subirán por error carpetas temporales pesadas de C# como bin/ y obj/. Por lo tanto, se debe crear un archivo .gitignore en la raíz C:\Visual Studio Projects\NucleoBoard_LedControl\.gitignore con este contenido esencial.

Cree el archivo en blanco a través del siguiente comando (figura 20):

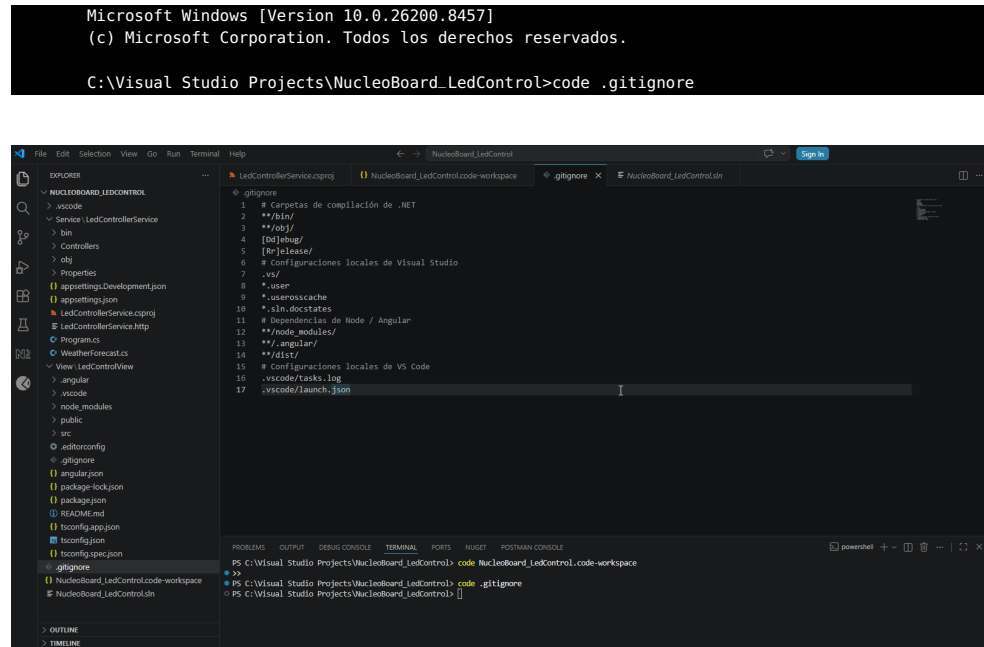


Figura 20: Archivo .gitignore en la raíz del proyecto.

Listing 1: Ejemplo de archivo de exclusiones

```
1 # Carpetas de compilacion de .NET
2 **/bin/
3 **/obj/
4 [Dd]ebug/
5 [Rr]elease/
6
7 # Configuraciones locales de Visual Studio
8 .vs/
9 *.user
10 *.userosscache
11 *.sln.docstates
12
13 # Dependencias de Node / Angular
14 **/node_modules/
15 **/.angular/
16 **/dist/
17
18 # Configuraciones locales de VS Code
19 .vscode/tasks.log
20 .vscode/launch.json
```

Para gestionar cómodamente las carpetas /Service y /View de forma independiente en una misma ventana de VS Code:

Ve al menú superior **File** ▶ **Open Workspace from File...** y seleccione este archivo (figura 21).

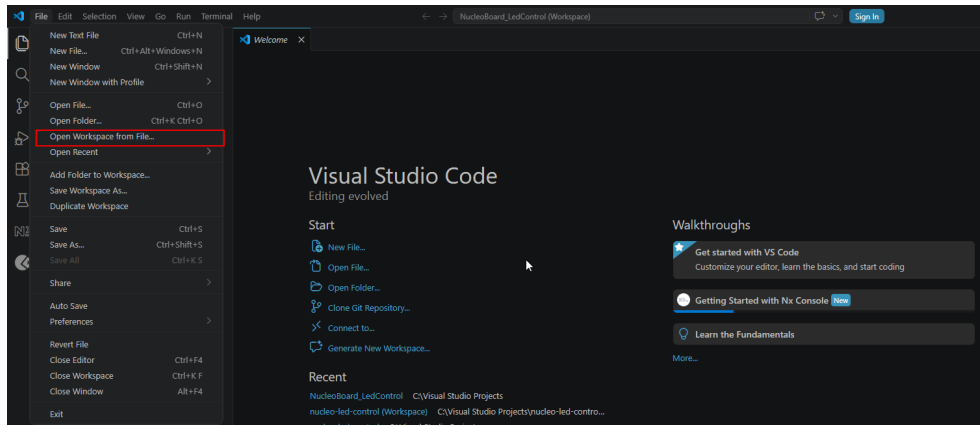


Figura 21: Abrir un archivo workspace.

Esto reestructurará tu barra lateral organizando de mnnera limpia ambos proyectos (figura 22).

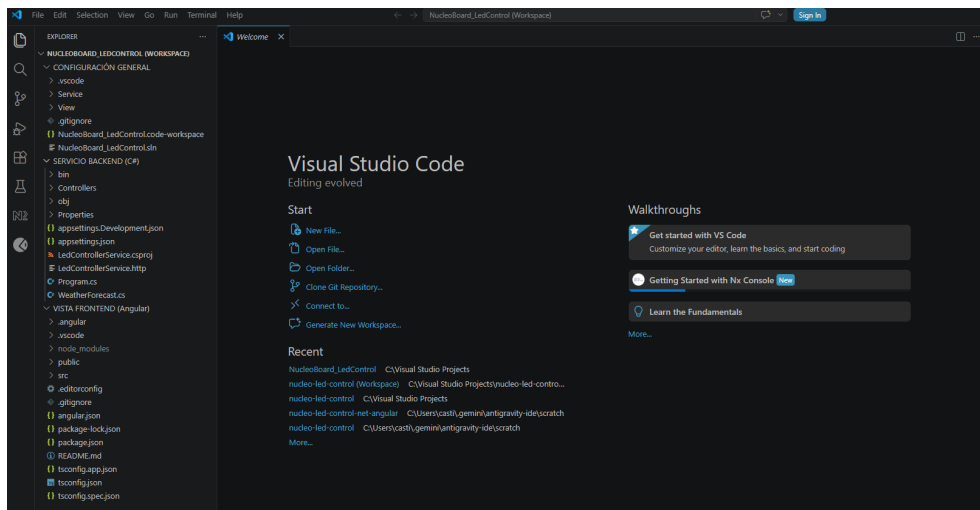


Figura 22: Reestructuración del proyecto.

6 COMPILACIÓN Y EJECUCIÓN DEL PROYECTO

6.1 Compilación y ejecución del backend

Abre una terminal y dirígete a la siguiente ruta:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl>cd Service\LedControllerService
```

Compile el proyecto para asegurar que no haya errores de namespaces:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl\Service\LedControllerService> dotnet build
```

Inicia el servidor de C#:

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl\Service\LedControllerService>dotnet run
```

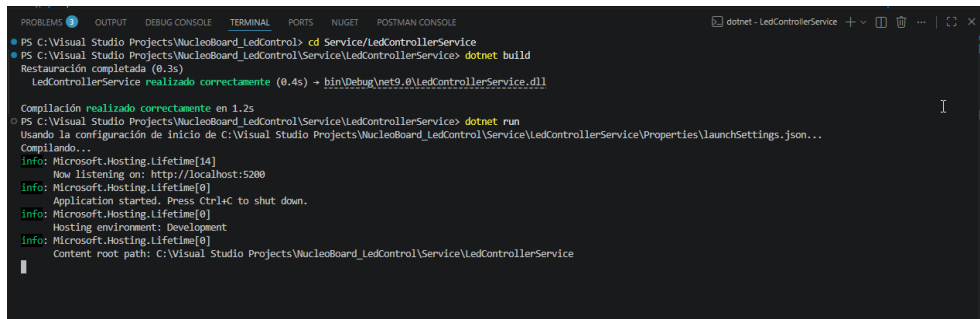


Figura 23: Compilación y ejecución del backend.

6.2 Compilación y ejecución del frontend

Abre otra pestaña en la terminal y ve al frontend

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl>cd View/LedControlView
```

Arranca el servidor de desarrollo en Angular

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl\View\LedControlView>npm start
```

7 TITLE

7.1 Crear el Servicio Serial (SerialService)

El archivo `SerialService` se encargará de gestionar directamente la comunicación física con la tarjeta STM32 Nucleo. Este servicio se registrará como un **Singleton** (una instancia única en memoria), debido a que el puerto COM físico sólo puede estar abierto por un único proceso a la vez. Este archivo realizará lo siguiente:

ABRIR Y CERRAR LA CONEXIÓN con el puerto COM seleccionado por el usuario y a la velocidad indicada.

AUTO CONEXIÓN INTELIGENTE : Guardará los datos del último puerto exitoso en un archivo local `serial-settings.json`. Si al arrancar el servidor detecta que la placa está conectada en ese puerto, se conectará de inmediato.

LECTURA ASÍNCRONA (BACKGROUND THREAD) : Creará un hilo de ejecución secundario (Thread) dedicado únicamente a escuchar datos provenientes de la tarjeta en segundo plano. Esto evita que el servidor Web API se congele mientras espera los datos del puerto serie.

NOTIFICACIONES EN TIEMPO REAL Utiliza **SignalR** (`IHubContext<SerialHub>`) para "empujar" de forma reactiva los datos recibidos (RX), transmitidos (TX) y el estado del LED hacia el frontend, sin que el cliente tenga que estar preguntando periódicamente (polling).

Dentro del editor de Visual Studio Code, ve a la carpeta `Service/LedControllerService`

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

PS C:\Visual Studio Projects\NucleoBoard_LedControl>cd Service/LedControllerService
```

Creas una nueva carpeta llamada **Services** (en plural).

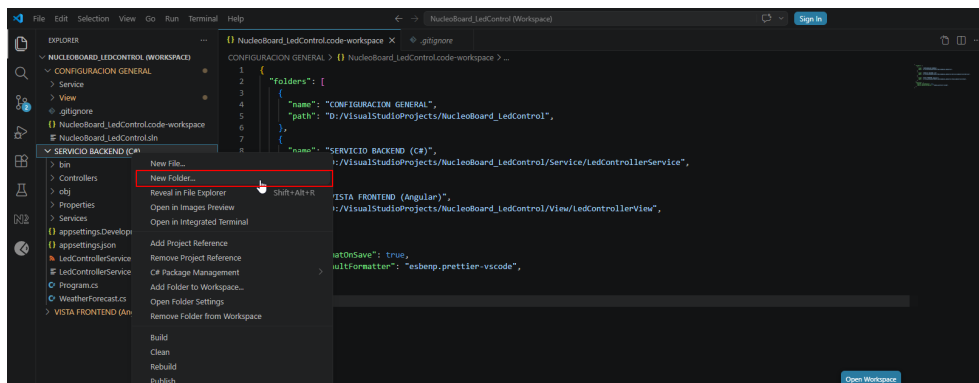


Figura 24: Creación de la carpeta **Services**.

Dentro de la carpeta **Services**, crea un archivo llamado **SerialService.cs**

```
1 using LedControllerService.Hubs;
2 using Microsoft.AspNetCore.SignalR;
3 using System;
4 using System.IO;
5 using System.IO.Ports;
6 using System.Text.Json;
7 using System.Threading;
8 using System.Threading.Tasks;
```

7.1.1 Namespaces

En C# (y otros lenguajes como C++ o Java), la palabra clave `using` se utiliza para importar Namespaces. Los Namespaces pueden visualizarse como carpetas organizadoras llenas de código escrito por ti o por terceros. En lugar de escribir el nombre completo de una clase cada vez (como `System.IO.Ports.SerialPort`), se coloca `using` al inicio y se puede escribir simplemente como `SerialPort`.

1. `using LedControllerService.Hubs;`

SIGNIFICADO Importa la carpeta virtual de nuestro propio proyecto donde declararemos el canal del WebSocket de SignalR.

USO Permite que este servicio haga referencia al canal `SerialHub` para poder enviarle mensajes a los navegadores web.

2. `using Microsoft.AspNetCore.SignalR;`

SIGNIFICADO Importa herramientas oficiales de Microsoft para la comunicación en tiempo real (SignalR).

USO Nos da acceso a la interfaz `IHubContext`, la cual permite que una clase de C# común y corriente (como este servicio) tome el control de las conexiones WebSockets activas para enviarles datos.

3. `using System;`

SIGNIFICADO El espacio de nombres base y fundamental de todo .NET.

USO Da soporte a tipos y objetos básicos de programación.

4. `using System.IO;`

SIGNIFICADO Significa *System Input/Output* (Entrada/Salida de archivos del sistema)

USO Nos permite interactuar con el disco duro de la PC para saber si existe el archivo de configuración (`File.Exists`), leerlo (`File.ReadAllText`) y escribir en él (`File.WriteAllText`)

5. `using System.IO.Ports;`

SIGNIFICADO El paquete que se instaló mediante NuGet para gestionar puertos serie.

USO Es el que hace toda la "magia" del hardware. Nos provee la clase `SerialPort`, que es el controlador de bajo nivel para detectar puertos USB conectados, abrir la comunicación, fijar baudios y transmitir/recibir texto.

6. `using System.Text.Json;`

SIGNIFICADO El serializador JSON oficial de Microsoft

USO Convierte la información de configuración en texto formateado JSON para guardarla en el archivo de texto `JsonSerializer.Serialize` y la vuelve a transformar en datos estructurados de C# cuando arranca la app `JsonSerializer.Deserialize`.

7. `using System.Threading;`

SIGNIFICADO Herramientas para la gestión de Hilos de ejecución (*Threads*).

USO Sirve para crear el hilo paralelo que lee la tarjeta continuamente y usar pausas breves (`Thread.Sleep`) para que el procesador de la PC descance cuando no hay datos entrantes.

8. `using System.Threading.Tasks;`

SIGNIFICADO Librería para programación asíncrona moderna de .NET

uso Nos provee la clase Task, que sirve para ejecutar pequeñas tareas en segundo plano sin interrumpir el flujo principal (como cuando le enviamos la pregunta ? a la placa medio segundo después de haber conectado.)

Las siguientes líneas son los cimientos de la Programación Orientada a Objetos (POO) en C# y definen la identidad de nuestro archivo.

```

1 namespace LedControllerService.Services
2 {
3     public class SerialService
4     {
5
6     }
7 }
```

1. namespace LedControllerService.Services

Un namespace es un contenedor lógico que sirve para organizar las clases y evitar conflictos de nombres (por ejemplo, si se crea una clase llamada Logger) y Microsoft tiene otra igual, los namespaces evitarán que choquen.

- LedControllerService.Services le dice al compilador de C#: *"Esta clase pertenece al proyecto LedControllerService y está guardada dentro del grupo lógico Services"*
- Por convención, los namespaces reflejan la estructura física de tus carpetas en el disco duro.

2. public

Es un **modificador de acceso**. Define quién tiene permitido ver y utilizar esta clase. Significa que **cualquier otra clase** del proyecto (como tu controlador SerialController o el archivo de inicio Program.cs) puede instanciar, leer y ejecutar este servicio. Si fuera private (privado) o internal (interno), su acceso estaría muy restringido.

3. class

Una clase es la plantilla o molde a partir del cual creamos objetos en programación. La palabra clave class le dice a C# que a continuación definiremos una estructura que contendrá datos (variables como el puerto COM actual) y comportamientos (métodos como Connect o Disconnect).

4. SerialService

Es simplemente el nombre que le asignamos a esta clase.

- Por convención en C#, los nombres de las clases se escriben en PascalCase (comienzan con mayúscula y cada nueva palabra empieza con mayúscula) y deben ser descriptivos de su función. En este caso, indica que es un "Servicio" encargado de la comunicación "Serial".

7.1.2 Declaración de variables

Estas declaraciones representan las propiedades y estados internos de nuestro servicio. Se les conoce como campos privados (private fields) y definen los datos que el servicio necesita recordar y utilizar para funcionar.

Antes de ver a detalle cada variable, es importante recordar estos tres puntos claves de la sintaxis en C#:

- **private** (privado): Significa que estas variables sólo existen dentro de esta clase. Ninguna clase externa puede verlas ni modificarlas directamente.
- **El guión bajo (_)**: Es una convención estándar en C# para identificar rápidamente que una variable es un campo privado de la clase.

- El signo de interrogación (?): Por ejemplo, `SerialPort?` o `string?` indica que la variable es **nuleable** (puede valer **null**, si, por ejemplo, la tarjeta está desconectada).

```

1 private readonly IHubContext<SerialHub> _hubContext;
2 private readonly ILogger<SerialService> _logger;
3 private SerialPort? _serialPort;
4 private string? _currentPort;
5 private int _currentBaudRate = 115200;
6 private string? _savedPort;
7 private int _savedBaudRate = 115200;
8 private Thread? _readThread;
9 private bool _isRunning;
10 private readonly object _lock = new();
11 private readonly string _settingsPath;

```

1. Comunicación en tiempo real y logs

- `private readonly IHubContext<SerialHub> _hubContext;`
QUÉ ES: La interfaz para interactuar con nuestro hub de SignalR (WebSockets).
USO Es la que nos permite hacer `_hubContext.Clients.All.SendAsync(...)` para enviar datos de forma instantánea a los navegadores.
- `private readonly ILogger<SerialService> _logger;`
QUÉ ES: El grabador de logs del sistema de ASP.NET Core.
USO Escribe mensajes en la consola negra del servidor (p.e. `_logger.LogInformation("Puerto conectado")`) para que pueda monitorear en tiempo real qué hace el backend.

2. Control de la conexión física

- `private SerialPort? _serialPort;`
QUÉ ES: El objeto físico que controla el puerto COM.
USO: Contiene los métodos nativos del sistema para abrir, cerrar, leer y escribir datos en el cable USB. Vale **null** cuando no hay conexión.
- `private string? _currentPort;`
QUÉ ES: El nombre del puerto activo, por ejemplo, "COM3".
- `private int _currentBaudRate = 115200;`
QUÉ ES: La velocidad de comunicación activa del puerto (115200 bits por segundo por defecto).

3. Recordar la última conexión (Persistencia)

- `private string? _savedPort;`
- `private int? _savedBaudRate = 115200;`
QUÉ ES: Variables temporales que guardan en memoria el último puerto y baudios existentes cargados desde el archivo JSON de configuración.
- `private readonly string? _settingsPath;`
QUÉ ES: La ruta absoluta en el disco duro donde se creará y leerá el archivo de configuración `serial-settings.json`.

4. Hilos y control de concurrencia (Multithreading)

- `private Thread? _readThread;`
QUÉ ES: El objeto que representa nuestro hilo secundario de procesamiento.

USO: Ejecuta el bucle de lectura del puerto de forma aislada para no bloquear la página web principal.

- `private bool _isRunning;`

QUÉ ES: Una bandera booleana (verdadero/falso).

USO: Controla el bucle de lectura. Si es true, el hilo sigue leyendo; cuando es false (al desconectar), el hilo se apaga de forma limpia.

- `private readonly object _lock = new();`

QUÉ ES: Un objeto de bloqueo exclusivo.

USO: Como tenemos varios hilos de ejecución activos (el de lectura de datos, y el de la web que envía comandos como encender el LED), este objeto sirve de "semáforo" para asegurar que un sólo hilo a la vez intente comunicarse con la tarjeta STM32, evitando que el puerto colapse por peticiones simultáneas.

7.1.3 Constructor de la clase

El constructor es una función especial que se **ejecuta automáticamente una sola vez** en el momento exacto en que el servidor crea la clase en memoria. Su objetivo es inicializar los datos y preparar la clase para que pueda ser utilizada.

Se le reconoce porque **tiene el mismo nombre que la clase** y no devuelve ningún valor.

```

1 public SerialService(IHubContext<SerialHub> hubContext, ILogger<SerialService>
   logger)
2 {
3     _hubContext = hubContext;
4     _logger = logger;
5     // Ruta del archivo JSON para guardar las configuraciones de conexion
6     _settingsPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "serial-
       settings.json");
7     LoadSettings();
8
9     // Intenta auto-conectar al iniciar el servidor
10    TryAutoConnect();
11 }

```

Los parámetros `hubContext` y `logger` se reciben utilizando un patrón de diseño fundamental en ASP.NET Core llamado **Inyección de Dependencias**.

CÓMO FUNCIONA: En lugar de que nuestro código intente crear manualmente los servicios de SignalR o de Logs (escribiendo un `new HubContext(...)` lo cual sería muy complejo), el framework de .NET se encarga de crear esos servicios al arrancar el servidor.

LA INYECCIÓN: Al declarar `IHubContext<SerialHub> hubContext` en los parámetros de entrada del constructor, le estamos diciendo a .NET "*necesito usar SignalR y Logs, por favor pásamelos cuando crees este servicio*". .NET los busca y los "inyecta" automáticamente en la función.

EL GUARDADO: Las primeras dos líneas

```

    \_hubContext = hubContext;
    \_logger = logger;

```

Toman esos servicios provistos por .NET y los guardan en nuestras variables privadas (los campos con guión bajo) para que cualquier otra función de la clase pueda usarlos más tarde.

El constructor de la clase

A nivel técnico, toda clase tiene siempre un constructor, pero no siempre tienes que escribirlo tú.

Si crea una clase y no escribes ningún constructor, el compilador de C# creará uno automáticamente tras bambalinas. A esto se le conoce como **Constructor por Defecto** (Default Constructor). Por ejemplo, si escribes esto

```
public class Persona
{
    public string Nombre { get; set; }
}
```

El compilador lo transforma en esto:

```
public class Persona
{
    public string Nombre { get; set; }

    //Creado automaticamente por C#
    public Persona()
    {
    }
}
```

Debes escribir manualmente un constructor en tres casos principales:

- **Para inyección de dependencias (como en nuestro proyecto):** Si tu clase necesita que .NET le pase herramientas externas al iniciar (como el contexto de SignalR o el Logger), debes escribir el constructor para declarar qué dependencias requieres.
- **Para inicializar datos obligatorios:** Si deseas que no se pueda crear una clase sin ciertos datos.
- **Para ejecutar código de preparación:** Si necesitas realizar acciones preias (como calcular rutas de archivos, conectar una base de datos o suscribir eventos.)

Función del cuerpo de la función

1. Calcular la ruta física del archivo de configuración

```
_settingsPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "
    serial-settings.json");
```

QUÉ HACE: `AppDomain.CurrentDomain.BaseDirectory` obtiene la ruta de la carpeta exacta donde se está ejecutando el backend en tu disco duro.

RESULTADO Crea la ruta absoluta segura en disco (por ejemplo `D:\VisualStudio Projects\NucleoBoard_LedControl\bin\Debug\net9.0\serial-settings.json`) para guardar configuraciones sin importar en qué computadora se ejecute.

2. Cargar la configuración previa

```
LoadSettings();
```

Llama al método interno `LoadSettings()`. Este método va al disco duro, abre el archivo `serial-settings.json` (si ya existe) y lee cuál fue la última velocidad de baudios y el último puerto COM con el que el usuario se conectó con éxito.

3. Autoconexión inteligente

```
TryAutoConnect();
```

Llama al método `TryAutoConnect()`. Si el paso anterior leyó un puerto COM válido (por ejemplo, COM3) y el sistema operativo detecta que la tarjeta STM32 está físicamente conectada en ese puerto en este momento, el constructor abre el puerto inmediatamente de forma automática sin esperar a que el usuario tenga que presionar el botón de "Conectar" en la página web.

7.1.4 Contenido de la clase

La siguiente función es sencilla pero fundamental: se encarga de consultar el sistema operativo Windows qué puertos COM físicos o virtuales están activos y utilizables en la computadora en ese instante.

```
public string[] GetAvailablePorts()
{
    return SerialPort.GetPortNames();
}
```

- `public`: Permite que clases externas (como nuestro controlador API `SerialController`) llamen a esta función para obtener la lista de puertos.
- `string[]`: Es el tipo de dato que devuelve la función. En este caso, una lista o "vector" de textos (por ejemplo: ["COM1", "COM3"]).
- `SerialPort.GetPortNames()`: Es un método nativo provisto por .NET (`System.IO.Ports`). Este método le pregunta a Windows: "Oye, ¿qué dispositivos seriales están conectados ahora mismo?".
- `return`: Envía la lista de puertos obtenida de vuelta a quien haya llamado a la función.

Cuando conectas tu tarjeta de desarrollo STM32 Nucleo mediante cable USB a la computadora, Windows le asigna automáticamente un puerto serie virtual. Esta función permite que, cuando abras la página web del panel de control en tu navegador, el menú desplegable (Select) se llene automáticamente con los puertos reales disponibles en tu PC, evitando que tengas que escribir a mano a qué puerto conectarte. Si no hay nada conectado, devolverá una lista vacía.

```
1 public object GetStatus()
2 {
3     lock (_lock)
4     {
5         return new
6         {
7             IsConnected = _serialPort?.IsOpen ?? false,
8             PortName = _serialPort?.IsOpen == true ? _currentPort : null,
9             BaudRate = _serialPort?.IsOpen == true ? _currentBaudRate : 0,
10            SavedPort = _savedPort,
11            SavedBaudRate = _savedBaudRate
12        };
13    }
14 }
```

Esta función se encarga de recopilar e informar el estado actual de la conexión serie y la configuración del servidor, empaquetándolo todo en un formato que el navegador pueda entender fácilmente.

1. El tipo de retorno object

En C#, object es la clase base de la cual heredan todos los tipos de datos. Retornar un objeto object nos permite crear y devolver un **Objeto Anónimo** (un objeto "al vuelo", sin necesidad de crear una clase formal para él).

Cuando una respuesta es enviada al navegador a través de nuestra API, .NET la convierte automáticamente en un formato JSON limpio y estructurado.

2. El semáforo lock (_lock)

Dado que el puerto serie puede estar siendo modificado por varios procesos al mismo tiempo (por ejemplo, el hilo de lectura secundario o una orden de desconexión repentina), la instrucción lock (_lock) detiene temporalmente cualquier otra operación sobre el puerto mientras recopilamos el estado. Esto evita errores de concurrencia y caídas del servidor.

3. Las propiedades que devuelve (lógica paso a paso)

- `IsConnected = _serialPort?.IsOpen ?? false`
 - *Lógica:* Usa dos operadores modernos de C#:
 - * El operador nulo-condicional (?): Si SerialPort es nulo (no existe), devuelve nulo en lugar de lanzar un error.
 - * El operador de coalescencia nula (??): Si la parte izquierda es nula, toma el valor de la derecha, false.
 - *Resultado:* Devuelve true si el puerto está físicamente conectado y abierto, o false si no lo está.
- `PortName = _serialPort?.IsOpen == true ? _currentPort : null`
 - *Lógica:* Es un operador ternario (condición ? si_es_verdadero : si_es_falso).
 - *Resultado:* Si el puerto está abierto, devuelve el nombre del puerto activo (ej: "COM3"). Si está cerrado, devuelve null.
- `BaudRate = _serialPort?.IsOpen == true ? _currentBaudRate : 0`
 - *Resultado:* Si está conectado, devuelve la velocidad actual (ej: 115200). Si no, devuelve 0.
- `SavedPort = _savedPort y SavedBaudRate = _savedBaudRate`
 - *Resultado:* Devuelve la última configuración de puerto y baudios que se guardó en el archivo JSON. Esto sirve para que el frontend sepa qué puerto autoseleccionar por defecto en la pantalla al cargar la página.

Tenga en cuenta que las variables:

```
IsConnected
PortName
BaudRate
SavedPort
SavedBaudRate
```

nunca se declararon antes. Estas se declaran e inicializan en ese preciso instante. En C# esto se conoce como **Tipos Anónimos (Anonymous Types)**.

¿Qué es un Tipo Anónimo?

Cuando se utiliza la palabra clave `new` seguida de llaves `{...}` sin especificar el nombre de una clase, le estás diciendo a C#: *"por favor crea una clase temporal en segundo plano, asígnele propiedades con estos nombres, y deduce sus tipos de datos según los valores que les estoy asignando a la derecha"*.

Tipos Anónimos

En este fragmento de código:

```
return new
{
    IsConnected = _serialPort?.IsOpen ?? false, // C# deduce que es un
    booleano (bool)
    PortName = _serialPort?.IsOpen == true ? _currentPort : null, // C#
    deduce que es un texto (string)
    BaudRate = _serialPort?.IsOpen == true ? _currentBaudRate : 0, // C#
    deduce que es un entero (int)
    SavedPort = _savedPort,
    SavedBaudRate = _savedBaudRate
};
```

El compilador crea en tiempo de compilación una clase temporal oculta que contiene exactamente este aspecto:

```
public class ClaseAnonimaOculta
{
    public bool IsConnected { get; }
    public string PortName { get; }
    public int BaudRate { get; }
    public string SavedPort { get; }
    public int SavedBaudRate { get; }
}
```

¿Por qué hacemos esto en lugar de crear una nueva clase?

AHORRO DE CÓDIGO : Nos evita la molestia de tener que crear un archivo o clase llamada `EstadoSerial.cs` que sólo se usaría una vez en todo el proyecto para esta respuesta.

COMPATIBILIDAD CON JSON : El servidor de ASP.NET Core lee las propiedades de este objeto anónimo y las convierte automáticamente en un formato JSON estándar que viaja hacia Angular:

```
{
    "isConnected": true,
    "portName": "COM3",
    "baudRate": 115200,
    "savedPort": "COM3",
    "savedBaudRate": 115200
}
```

Es una característica muy utilizada en .NET para construir respuestas rápidas de APIs.

Esta es la función más importante del servicio para la conexión física. Se encarga de abrir la compuerta de comunicación con la tarjeta, configurar el canal de datos y encender el hilo de escucha en segundo plano

```

1 // Abre la conexión con el puerto serie físico
2 public bool Connect(string portName, int baudRate, out string errorMessage)
3 {
4     lock (_lock)
5     {
6         errorMessage = string.Empty;
7         if (_serialPort != null && _serialPort.IsOpen)
8         {
9             Disconnect();
10        }
11        try
12        {
13            _serialPort = new SerialPort(portName, baudRate)
14            {
15                ReadTimeout = 1000,
16                WriteTimeout = 1000,
17                NewLine = "\n" // Sincroniza las lecturas por caracteres de
                salto de línea
18            };
19            _serialPort.Open();
20            // Limpiar buffers iniciales para descartar basura de conexión
21            _serialPort.DiscardInBuffer();
22            _serialPort.DiscardOutBuffer();
23            _currentPort = portName;
24            _currentBaudRate = baudRate;
25            _savedPort = portName;
26            _savedBaudRate = baudRate;
27
28            SaveSettings(portName, baudRate);
29            // Inicializa el hilo de lectura secundario
30            _isRunning = true;
31            _readThread = new Thread(ReadLoop)
32            {
33                IsBackground = true,
34                Name = "SerialReadThread"
35            };
36            _readThread.Start();
37            _logger.LogInformation($"Puerto serie {portName} abierto con éxito a
                {baudRate} bps.");
38
39            // Notificar a los clientes web a través de SignalR
40            _hubContext.Clients.All.SendAsync("ReceiveStatus", new { IsConnected
                = true, PortName = portName, BaudRate = baudRate });
41            _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"[Sistema] Puerto
                COM conectado con éxito a {baudRate} bps.");
42
43            // Preguntar proactivamente el estado del LED a la placa ('?') tras
                conectar
44            Task.Run(async () =>
45            {
46                await Task.Delay(500);
47                SendCommand("?");
48            });
49            return true;
50        }
51        catch (Exception ex)
52        {

```

```

53     errorMessage = ex.Message;
54     _logger.LogError($"Error al abrir el puerto {portName}: {ex.Message}
");
55     _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"[Sistema] Error
de conexion en {portName}: {ex.Message}");
56     return false;
57 }
58 }
59 }

```

1. Inicialización del método y el parámetro out

```

1 public bool Connect(string portName, int baudRate, out string
errorMessage)

```

- Retorna **bool**: Devuelve **true** si la conexión fue exitosa y **false** si falló.
- **out string** errorMessage: El modificador **out** en C# permite que una función devuelva más de un valor. En este caso, si la función retorna **false**, "saca" el mensaje de error exacto dentro de la variable errorMessage para que el controlador pueda mostrárselo al usuario en la página web.

2. Evitar conexiones duplicadas y proteger concurrencia.

```

1 lock (_lock)
2 {
3     errorMessage = string.Empty;
4     if (_serialPort != null && _serialPort.IsOpen)
5     {
6         Disconnect();
7     }
8 }

```

- **lock** (_lock): Bloquea el acceso al puerto. Si el usuario hace clic rápido dos veces en "Conectar", esto evita que se intenten abrir dos conexiones al mismo tiempo.
- **Disconnect()**: Si ya había una conexión activa (o colgada en memoria), primero se cierra y libera limpiamente antes de intentar abrir la nueva.

3. Configuración y apertura del puerto

```

1 _serialPort = new SerialPort(portName, baudRate)
2 {
3     // Espera 1 seg para leer antes de dar timeout
4     ReadTimeout = 1000,
5     // Espera 1 seg para escribir antes de dar timeout
6     WriteTimeout = 1000,
7     // Define que cada mensaje del uC termina con un salto de linea
8     NewLine = "\n"
9 };
10 _serialPort.Open(); // Abre fisicamente la conexion
11
12 // Limpiar buffers para evitar ruido de conexion inicial
13 _serialPort.DiscardInBuffer();
14 _serialPort.DiscardOutBuffer();
15

```

- Crea el objeto SerialPort con los parámetros del usuario y define los tiempos límite de espera (timeouts).

- `DiscardInBuffer()` y `DiscardOutBuffer()`: Limpia toda la "basura" o ruido eléctrico que pueda haber quedado en el cable USB en conexiones anteriores para iniciar con un canal limpio de datos.

4. Guardar Estado y persistencia

```

1      _currentPort = portName;
2      _currentBaudRate = baudRate;
3      _savedPort = portName;
4      _savedBaudRate = baudRate;
5      // Guarda los datos en serial-settings.json
6      SaveSettings(portName, baudRate);

```

- Actualiza las variables de estado en memoria y escribe en el disco duro para recorrer este puerto en la siguiente autoconexión.

5. Encender el Hilo secundario de lectura

```

1      _isRunning = true;
2      _readThread = new Thread(ReadLoop)
3      {
4          // Indica que el hilo morirá automáticamente si la aplicación
           principal se apaga
5          IsBackground = true,
6          Name = "SerialReadThread" // Nombre descriptivo para depurar
7      };
8      _readThread.Start(); // Arranca el bucle 'ReadLoop'

```

- **Multithreading:** Aquí es donde se delega la lectura a un carril independiente. El bucle `ReadLoop` se ejecutará en paralelo, escuchando lo que la tarjeta STM32 tenga que decir, sin congelar las peticiones de los usuarios en la web.

6. Avisar en tiempo real mediante SignalR

```

1      _hubContext.Clients.All.SendAsync("ReceiveStatus", new {
           IsConnected = true, PortName = portName, BaudRate = baudRate });
2      _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"[Sistema]
           Puerto COM conectado con éxito a {baudRate} bps.");

```

- Envía notificaciones de manera instantánea a todos los navegadores conectados diciendo: "*¡Ya estamos conectados!*" y añade un registro de log a la consola.

7. Sincronización Automática con la placa (LED)

```

1      Task.Run(async () =>
2      {
3          // Espera 0.5s para que la electrónica se estabilice
4          await Task.Delay(500);
5          // Envía el comando de pregunta a la placa
6          SendCommand("?");
7      }

```

- `Task.Run`: Inicia una tarea asíncrona de fondo. Espera medio segundo (para que la comunicación serial recién abierta se estabilice) y envía automáticamente el carácter '?' a la placa Nucleo. La placa responderá si el LED está encendido o apagado y la web se actualizará sola.

8. Captura de Errores (Try-Catch)

```

1      catch (Exception ex)
2      {
3          errorMessage = ex.Message; // Guarda el error fisico (ej. "
          Acceso denegado" o "El puerto no existe")
4          _logger.LogError(...);
5          _hubContext.Clients.All.SendAsync(...);
6          return false;
7      }
8

```

- Si algo falla (ej. el puerto está siendo usado por otro programa como STM32CubeIDE o el cable está desconectado), se captura el error, se notifica al usuario en la consola web, y la función devuelve false.

Esta función se encarga de realizar la desconexión controlada y la liberación de recursos del hardware en tu computadora. Cuando la comunicación con la tarjeta termina, no basta con "dejar de usar" el puerto; es necesario avisarle al sistema operativo de forma limpia para que el puerto COM no quede bloqueado o "colgado".

```

1 // Cierra la conexion y libera los recursos del puerto
2 public void Disconnect()
3 {
4     lock (_lock)
5     {
6         _isRunning = false;
7
8         if (_serialPort != null)
9         {
10            try
11            {
12                if (_serialPort.IsOpen)
13                {
14                    _serialPort.Close();
15                }
16            }
17            catch (Exception ex)
18            {
19                _logger.LogError($"Error al cerrar el puerto serie: {ex.Message}
20                ");
21            }
22            finally
23            {
24                _serialPort.Dispose();
25                _serialPort = null;
26            }
27        }
28        _currentPort = null;
29        _logger.LogInformation("Puerto serie desconectado.");
30        _hubContext.Clients.All.SendAsync("ReceiveStatus", new { IsConnected =
31        false, PortName = (string?)null, BaudRate = 0 });
32        _hubContext.Clients.All.SendAsync("ReceiveTxLog", "[Sistema] Puerto
33        serie desconectado.");
34    }
35 }
36

```

1. El semáforo y la señal de parada

```

1     lock (_lock)
2     {
3         _isRunning = false;
4     }
5

```

- `lock (_lock)`: Protege el método. Asegura que nadie intente enviar un comando a la placa ni consultar el estado mientras se está desmantelando la conexión.
- `_isRunning = false`: Es el interruptor de apagado. Al cambiar esta variable a `false`, le estamos diciendo al hilo de lectura en segundo plano (`ReadLoop`) que **debe terminar su bucle de inmediato** y apagarse de forma segura.

2. Cierre seguro del puerto físico

```

1      if (_serialPort != null)
2      {
3          try
4          {
5              if (_serialPort.IsOpen)
6              {
7                  _serialPort.Close(); // 1. Cierra el canal fisico
8              }
9          }
10         catch (Exception ex)
11         {
12             _logger.LogError($"Error al cerrar el puerto serie: {ex.
13             Message}");
14         }

```

- primero verifica si la variable `_serialPort` existe (no es nula).
- Si el puerto está abierto, llama a `_serialPort.Close()`. Esto corta la conexión eléctrica virtual con el puerto USB. Si por alguna razón de hardware esto falla, el `catch` captura la excepción para evitar que el servidor web se caiga.

3. Liberación absoluta de memoria (finally)

```

1      finally
2      {
3          // 2. Libera recursos a nivel de Sistema Operativo
4          _serialPort.Dispose();
5          // 3. Destruye la referencia en memoria C#
6          _serialPort = null;
7      }
8  }
9

```

- El bloque `finally` se ejecuta **sí o sí**, haya habido errores en el paso anterior o no.
- `_serialPort.Dispose()`: Desecha el objeto y le dice a Windows: "He terminado, puedes liberar por completo los controladores de este puerto COM para que otros programas (como *STM32CubeIDE*) puedan usarlo".
- `_serialPort = null;`: Deja la variable limpia en `null` indicando que no hay ningún puerto enlazado.

4. Limpieza de estado local y Notificación Web instantánea

```

1  _currentPort = null;
2  _logger.LogInformation("Puerto serie desconectado.");
3
4  _hubContext.Clients.All.SendAsync("ReceiveStatus", new { IsConnected =
    false, PortName = (string?)null, BaudRate = 0 });

```

```
5 _hubContext.Clients.All.SendAsync("ReceiveTxLog", "[Sistema] Puerto serie
    desconectado.");
```

- Resetea el nombre del puerto activo a null;
- Mediante SignalR, transmite a todos los navegadores web que el servidor está Desconectado (`isConnected = false`) y envía un mensaje a la consola de logs para que la interfaz cambie sus botones a estado de desconectado inmediatamente.

Esta función es la encargada de la transmisión de datos (TX). Es el puente que toma una orden que viene desde el botón de la página web (como "0", "1", o CR o LF) y la inyecta directamente por el cable USB hacia los pines físicos del microcontrolador STM32.

```
1 public bool SendCommand(string cmd)
2 {
3     lock (_lock)
4     {
5         if (_serialPort == null || !_serialPort.IsOpen)
6         {
7             _logger.LogWarning("Intento de envio de comando sin puerto serie
            activo.");
8             return false;
9         }
10
11         try
12         {
13             _serialPort.Write(cmd);
14             _logger.LogInformation($"Enviado comando serie: '{cmd}'");
15             _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"Enviado: '{cmd}'
            ");
16             return true;
17         }
18         catch (Exception ex)
19         {
20             _logger.LogError($"Error al transmitir comando serie: {ex.Message}");
21             _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"Error al enviar:
            {ex.Message}");
22             return false;
23         }
24     }
25 }
```

1. Inicialización del método

```
1 public bool SendCommand(string cmd)
2 {
3     lock (_lock)
4     {
```

- Retorno bool: Devuelve true si el comando se envió con éxito y false si no se pudo transmitir.
- lock (_lock): Al igual que la conexión y desconexión, este cerrojo garantiza que no haya dos comandos chocando en el puerto serie al mismo tiempo (por ejemplo, si el usuario da múltiples clics rápidos en la interfaz).

2. Verificación

```

1  if (_serialPort == null || !_serialPort.IsOpen)
2  {
3      _logger.LogWarning("Intento de envio de comando sin puerto serie
      activo.");
4      return false;
5  }
6

```

- Antes de intentar enviar nada, verifica si el puerto serie está instanciado y abierto.
- Si el puerto está cerrado o es nulo (por ejemplo, si el usuario intenta encender el LED sin haber conectado el puerto COM antes), el programa escribe una advertencia en los logs del servidor y sale inmediatamente devolviendo false para evitar un fallo crítico o una excepción no controlada.

3. Transmisión del Comando y Actualización en Tiempo Real

```

1  try
2  {
3      _serialPort.Write(cmd); // Envio fisico de datos a la tarjeta
4      _logger.LogInformation($"Enviado comando serie: '{cmd}'");
5
6      // Registra la transmision en el panel web
7      _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"Enviado: '{cmd}'");
8
9      return true;
10 }

```

- `_serialPort.Write(cmd)`: Esta línea de código toma la cadena cmd y la transmite eléctricamente a través del canal TX del cable de comunicación
- SignalR ReceiveTxLog: Envía un mensaje instantáneo a la interfaz gráfica del usuario con el texto "Enviado: '1'" o "Enviado: '0'", permitiendo que el log del cliente web muestre la acción inmediatamente en su pantalla.

4. Control de fallos

```

1  catch (Exception ex)
2  {
3      _logger.LogError($"Error al transmitir comando serie: {ex.Message}
      ");
4      _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"Error al
      enviar: {ex.Message}");
5      return false;
6  }
7

```

- Si ocurre un error de hardware en pleno envío (por ejemplo, si desconectas el cable USB de la tarjeta justo en el momento en que presionas el interruptor en la página web), el bloque catch captura la excepción, informa del error al servidor, avisa a la interfaz mediante SignalR de que el comando falló, y devuelve false.

Este bloque de código es el encargado de recepción de datos (**RX**) del servidor. Se ejecuta dentro del hilo secundario (Thread) independiente. Su único trabajo es estar al acecho, esperando a que el microcontrolador STM32 envíe algún texto por el puerto serie para leerlo, interpretarlo y mandarlo inmediatamente a tu navegador web.

```

1 private void ReadLoop()
2 {
3     while (_isRunning)
4     {
5         try
6         {
7             if (_serialPort != null && _serialPort.IsOpen)
8             {
9                 string line = _serialPort.ReadLine();
10                if (!string.IsNullOrEmpty(line))
11                {
12                    string trimmedLine = line.Trim();
13                    _logger.LogInformation($"Recibido del microcontrolador: \"{
trimmedLine}\"");
14
15                    // Envía la línea cruda de log a la consola del frontend
16                    _hubContext.Clients.All.SendAsync("ReceiveData", trimmedLine
);
17
18                    // Detecta el estado del LED para actualizar la gráfica y
foco en la web
19                    string lowerLine = trimmedLine.ToLower();
20                    if (lowerLine.Contains("encendido"))
21                    {
22                        _hubContext.Clients.All.SendAsync("ReceiveLedState",
true);
23                    }
24                    else if (lowerLine.Contains("apagado"))
25                    {
26                        _hubContext.Clients.All.SendAsync("ReceiveLedState",
false);
27                    }
28                }
29            }
30            else
31            {
32                Thread.Sleep(100);
33            }
34        }
35        catch (TimeoutException)
36        {
37            // Ignora timeouts de espera de lectura normales
38        }
39        catch (Exception ex)
40        {
41            _logger.LogError($"Error en el bucle de lectura del puerto serie: {
ex.Message}");
42
43            // Maneja la desconexión física inesperada (ej. desconectar cable
USB)
44            lock (_lock)
45            {
46                if (_serialPort == null || !_serialPort.IsOpen)
47                {
48                    HandleUnexpectedDisconnection();

```

```

49         break;
50     }
51 }
52 Thread.Sleep(500);
53 }
54 }
55 }

```

1. El ciclo de vida del hilo (while (!_isRunning))

```

1     while (!_isRunning)
2     {

```

El hilo ejecuta un ciclo constante mientras la variable `_isRunning` sea verdadera (true). Cuando el usuario presiona "Desconectar" en la web, `_isRunning` se vuelve false, lo que hace que este bucle termine limpiamente y el hilo se apague sin consumir recursos.

2. Lectura bloqueante y limpieza

```

1     string line = _serialPort.ReadLine();
2

```

- **Lectura por líneas:** El método `ReadLine()` es "bloqueante". Esto significa que el hilo secundario se queda "dormido" o pausado en esta línea exacta de código hasta que el microcontrolador envíe un mensaje que termine con un carácter de salto de línea (`\n`).
- **Limpieza de datos:** `line.Trim()` elimina espacios vacíos o caracteres invisibles de retorno de carro (`\r`) para quedarse solo con el texto crudo.

3. Transmisión a la Web y Detección de Estado

```

1     _hubContext.Clients.All.SendAsync("ReceiveData", trimmedLine);

```

- **Logs en tiempo real:** En cuanto llega una línea del microcontrolador (por ejemplo, "LED Encendido"), la retransmite inmediatamente a todos los clientes web conectados para que aparezca en la consola de logs del navegador.

```

1     string lowerLine = trimmedLine.ToLower();
2     if (lowerLine.Contains("encendido"))
3     {
4         _hubContext.Clients.All.SendAsync("ReceiveLedState", true);
5     }
6     else if (lowerLine.Contains("apagado"))
7     {
8         _hubContext.Clients.All.SendAsync("ReceiveLedState", false);
9     }
10

```

- **Sincronización Inteligente:** El hilo analiza el texto recibido. Si la tarjeta le envía "LED Encendido", el backend deduce que el pin físico está activo y le envía al frontend la orden de encender el LED virtual y la gráfica (`ReceiveLedState, true`). Lo mismo ocurre para "apagado".
- Esto asegura que la interfaz web esté siempre sincronizada con la realidad de la placa, incluso si el cambio de estado fue provocado por el propio microcontrolador al iniciar o reiniciar.

4. ¿Qué pasa si no hay conexión activa?

```

1     else
2     {
3         Thread.Sleep(100);
4     }

```

- Si la conexión aún no se ha abierto, el hilo se "duerme" por 100 milisegundos. Esto es crucial en multithreading: evita que la computadora consuma el 100% de la capacidad de tu procesador (CPU) girando en un bucle infinito a máxima velocidad.

5. Control de Excepciones y Desconexión Inesperada (Desenchufar cable)

A. Manejo de timeouts

```

1     catch (TimeoutException)
2     {
3         // Ignora timeouts de espera de lectura normales
4     }

```

- En la conexión definimos un tiempo de espera de 1 segundo (ReadTimeout = 1000). Si el microcontrolador pasa 1 segundo sin enviar nada, ReadLine() arroja un TimeoutException. El bucle atrapa este error y no hace nada, simplemente vuelve a empezar el bucle para seguir escuchando. Esto evita que el hilo se quede congelado para siempre si el microcontrolador está en silencio.

B. Error crítico/Desconexión física

```

1     catch (Exception ex)
2     {
3         _logger.LogError($"Error en el bucle de lectura: {ex.Message}"
4         );
5         lock (_lock)
6         {
7             if (_serialPort == null || !_serialPort.IsOpen)
8             {
9                 HandleUnexpectedDisconnection();
10                break; // Rompe el bucle while y apaga el hilo
11            }
12        }
13    }

```

- Si ocurre un error general (como un fallo de entrada/salida porque se desconectó físicamente el cable USB de la tarjeta):
- Captura la excepción y escribe el error en los logs del servidor.
- Activa el cerrojo lock (_lock) y verifica si el puerto serie efectivamente se cerró o se perdió.
- Llama a HandleUnexpectedDisconnection() para limpiar el estado y enviar una alerta de advertencia a la web de que se perdió la comunicación.
- Ejecuta un break para salir inmediatamente del bucle while, finalizando el hilo secundario de forma segura y sin dejar procesos colgados.

7.1.5 Código completo

```

1  using LedControllerService.Hubs;
2  using Microsoft.AspNetCore.SignalR;
3  using System;
4  using System.IO;
5  using System.IO.Ports;
6  using System.Text.Json;
7  using System.Threading;
8  using System.Threading.Tasks;
9  namespace LedControllerService.Services
10 {
11     public class SerialService
12     {
13         private readonly IHubContext<SerialHub> _hubContext;
14         private readonly ILogger<SerialService> _logger;
15         private SerialPort? _serialPort;
16         private string? _currentPort;
17         private int _currentBaudRate = 115200;
18         private string? _savedPort;
19         private int _savedBaudRate = 115200;
20         private Thread? _readThread;
21         private bool _isRunning;
22         private readonly object _lock = new();
23         private readonly string _settingsPath;
24         public SerialService(IHubContext<SerialHub> hubContext, ILogger<
SerialService> logger)
25         {
26             _hubContext = hubContext;
27             _logger = logger;
28             // Ruta del archivo JSON para guardar las configuraciones de
conexion
29             _settingsPath = Path.Combine(AppDomain.CurrentDomain.
BaseDirectory, "serial-settings.json");
30             LoadSettings();
31
32             // Intenta auto-conectar al iniciar el servidor
33             TryAutoConnect();
34         }
35         // Obtiene la lista de puertos COM disponibles en Windows (e.g. COM3
, COM4)
36         public string[] GetAvailablePorts()
37         {
38             return SerialPort.GetPortNames();
39         }
40         // Devuelve el estado actual de la conexion y las configuraciones
guardadas
41         public object GetStatus()
42         {
43             lock (_lock)
44             {
45                 return new
46                 {
47                     IsConnected = _serialPort?.IsOpen ?? false,
48                     PortName = _serialPort?.IsOpen == true ? _currentPort :
null,
49                     BaudRate = _serialPort?.IsOpen == true ?
_currentBaudRate : 0,
50                     SavedPort = _savedPort,
51                     SavedBaudRate = _savedBaudRate
52                 };

```

```

53     }
54 }
55 // Abre la conexion con el puerto serie fisico
56 public bool Connect(string portName, int baudRate, out string
errorMessage)
57 {
58     lock (_lock)
59     {
60         errorMessage = string.Empty;
61         if (_serialPort != null && _serialPort.IsOpen)
62         {
63             Disconnect();
64         }
65         try
66         {
67             _serialPort = new SerialPort(portName, baudRate)
68             {
69                 ReadTimeout = 1000,
70                 WriteTimeout = 1000,
71                 NewLine = "\n" // Sincroniza las lecturas por
caracteres de salto de linea
72             };
73             _serialPort.Open();
74             // Limpiar buffers iniciales para descartar basura de
conexion
75             _serialPort.DiscardInBuffer();
76             _serialPort.DiscardOutBuffer();
77             _currentPort = portName;
78             _currentBaudRate = baudRate;
79             _savedPort = portName;
80             _savedBaudRate = baudRate;
81
82             SaveSettings(portName, baudRate);
83             // Inicializa el hilo de lectura secundario
84             _isRunning = true;
85             _readThread = new Thread(ReadLoop)
86             {
87                 IsBackground = true,
88                 Name = "SerialReadThread"
89             };
90             _readThread.Start();
91             _logger.LogInformation($"Puerto serie {portName} abierto
con exito a {baudRate} bps.");
92
93             // Notificar a los clientes web a traves de SignalR
94             _hubContext.Clients.All.SendAsync("ReceiveStatus", new {
IsConnected = true, PortName = portName, BaudRate = baudRate });
95             _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"[
Sistema] Puerto COM conectado con exito a {baudRate} bps.");
96
97             // Preguntar proactivamente el estado del LED a la placa
('??') tras conectar
98             Task.Run(async () =>
99             {
100                 await Task.Delay(500);
101                 SendCommand("?");
102             });
103             return true;
104         }
105         catch (Exception ex)
106         {

```

```

107         errorMessage = ex.Message;
108         _logger.LogError($"Error al abrir el puerto {portName}:
{ex.Message}");
109         _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"[
Sistema] Error de conexion en {portName}: {ex.Message}");
110         return false;
111     }
112 }
113 }
114 // Cierra la conexion y libera los recursos del puerto
115 public void Disconnect()
116 {
117     lock (_lock)
118     {
119         _isRunning = false;
120
121         if (_serialPort != null)
122         {
123             try
124             {
125                 if (_serialPort.IsOpen)
126                 {
127                     _serialPort.Close();
128                 }
129             }
130             catch (Exception ex)
131             {
132                 _logger.LogError($"Error al cerrar el puerto serie:
{ex.Message}");
133             }
134             finally
135             {
136                 _serialPort.Dispose();
137                 _serialPort = null;
138             }
139         }
140         _currentPort = null;
141         _logger.LogInformation("Puerto serie desconectado.");
142         _hubContext.Clients.All.SendAsync("ReceiveStatus", new {
IsConnected = false, PortName = (string?)null, BaudRate = 0 });
143         _hubContext.Clients.All.SendAsync("ReceiveTxLog", "[Sistema]
Puerto serie desconectado.");
144     }
145 }
146 // Envia un comando de texto ('0', '1', '?') a la tarjeta
147 public bool SendCommand(string cmd)
148 {
149     lock (_lock)
150     {
151         if (_serialPort == null || !_serialPort.IsOpen)
152         {
153             _logger.LogWarning("Intento de envio de comando sin
puerto serie activo.");
154             return false;
155         }
156         try
157         {
158             _serialPort.Write(cmd);
159             _logger.LogInformation($"Enviado comando serie: '{cmd}'"
);

```

```

160         _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"
Enviado: '{cmd}'");
161         return true;
162     }
163     catch (Exception ex)
164     {
165         _logger.LogError($"Error al transmitir comando serie: {
ex.Message}");
166         _hubContext.Clients.All.SendAsync("ReceiveTxLog", $"
Error al enviar: {ex.Message}");
167         return false;
168     }
169 }
170 }
171 // Bucle de lectura que corre en el hilo secundario
172 private void ReadLoop()
173 {
174     while (!_isRunning)
175     {
176         try
177         {
178             if (_serialPort != null && _serialPort.IsOpen)
179             {
180                 string line = _serialPort.ReadLine();
181                 if (!string.IsNullOrEmpty(line))
182                 {
183                     string trimmedLine = line.Trim();
184                     _logger.LogInformation($"Recibido del
microcontrolador: \"{trimmedLine}\"");
185
186                     % Envia la linea cruda de log a la consola del
frontend
187                     _hubContext.Clients.All.SendAsync("ReceiveData",
trimmedLine);
188
189                     % Detecta el estado del LED para actualizar la
grafica y foco en la web
190                     string lowerLine = trimmedLine.ToLower();
191                     if (lowerLine.Contains("encendido"))
192                     {
193                         _hubContext.Clients.All.SendAsync("
ReceiveLedState", true);
194                     }
195                     else if (lowerLine.Contains("apagado"))
196                     {
197                         _hubContext.Clients.All.SendAsync("
ReceiveLedState", false);
198                     }
199                 }
200             }
201             else
202             {
203                 Thread.Sleep(100);
204             }
205         }
206         catch (TimeoutException)
207         {
208             // Ignora timeouts de espera de lectura normales
209         }
210         catch (Exception ex)
211         {

```

```

211         _logger.LogError($"Error en el bucle de lectura del
puerto serie: {ex.Message}");
212
213         // Maneja la desconexion fisica inesperada (ej.
desconectar cable USB)
214         lock (_lock)
215         {
216             if (_serialPort == null || !_serialPort.IsOpen)
217             {
218                 HandleUnexpectedDisconnection();
219                 break;
220             }
221         }
222         Thread.Sleep(500);
223     }
224 }
225
226 // Informa al frontend que el cable se desconecto
227 private void HandleUnexpectedDisconnection()
228 {
229     _logger.LogWarning("Desconexion inesperada de hardware detectada.
");
230     Disconnect();
231     _hubContext.Clients.All.SendAsync("ReceiveTxLog", "[Sistema]
Advertencia: Conexion perdida con la tarjeta STM32 (Puerto COM cerrado
inesperadamente).");
232 }
233 // Intenta recuperar la ultima conexion registrada
234 private void TryAutoConnect()
235 {
236     if (string.IsNullOrEmpty(_savedPort)) return;
237     string[] availablePorts = GetAvailablePorts();
238     bool isPortAvailable = Array.Exists(availablePorts, p => p.
Equals(_savedPort, StringComparison.OrdinalIgnoreCase));
239     if (isPortAvailable)
240     {
241         _logger.LogInformation($"Restableciendo conexion guardada de
forma automatica en el puerto: {_savedPort}");
242         string error;
243         Connect(_savedPort, _savedBaudRate, out error);
244     }
245     else
246     {
247         _logger.LogInformation($"El puerto guardado ({_savedPort})
no esta disponible. Esperando seleccion manual.");
248     }
249 }
250 // Carga configuraciones de serial-settings.json
251 private void LoadSettings()
252 {
253     try
254     {
255         if (File.Exists(_settingsPath))
256         {
257             string json = File.ReadAllText(_settingsPath);
258             var settings = JsonSerializer.Deserialize<SettingsData>(
json);
259             if (settings != null)
260             {
261                 _savedPort = settings.SavedPort;
262                 _savedBaudRate = settings.SavedBaudRate;

```

```

263     }
264 }
265 }
266 catch (Exception ex)
267 {
268     _logger.LogError($"No se pudo cargar la configuracion de
puerto: {ex.Message}");
269 }
270 }
271 // Guarda configuraciones en serial-settings.json
272 private void SaveSettings(string portName, int baudRate)
273 {
274     try
275     {
276         var settings = new SettingsData { SavedPort = portName,
SavedBaudRate = baudRate };
277         string json = JsonSerializer.Serialize(settings, new
JsonSerializerOptions { WriteIndented = true });
278         File.WriteAllText(_settingsPath, json);
279     }
280     catch (Exception ex)
281     {
282         _logger.LogError($"No se pudo guardar la configuracion de
puerto: {ex.Message}");
283     }
284 }
285 private class SettingsData
286 {
287     public string? SavedPort { get; set; }
288     public int SavedBaudRate { get; set; }
289 }
290 }
291 }

```

Listing 2: Servicio de comunicacion serie en C#

En este punto, el compilador nos mostrará que encontró 3 problemas en el archivo `SerialService.cs`, tal como se muestra en la figura 25. Esto es completamente normal, ya que aún no hemos creado el Hub de SignalR. Lo crearemos en el siguiente paso.

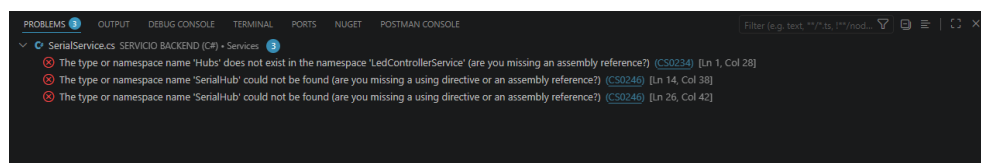


Figura 25: Errores encontrados en SerialService.cs.

7.2 Crear el Hub de SignalR (SerialHub.cs)

Un **Hub** (Concentrador) en SignalR es el canal lógico de comunicación donde se conectan los clientes externos (los navegadores web) a través de **WebSockets**. Funciona como un conmutador electrónico.

1. Dentro de Visual Studio Code, dirígete a la carpeta `Service` ▶ `LedControllerService`
2. Crea una nueva carpeta llamada **Hubs**
3. Dentro de la carpeta **Hubs**, crea un archivo llamado `SerialHub.cs`

4. Pega el siguiente código en el archivo

```

1 using Microsoft.AspNetCore.SignalR;
2 using System.Threading.Tasks;
3
4 namespace LedControllerService.Hubs
5 {
6     public class SerialHub : Hub
7     {
8         // Este Hub puede permanecer vacío ya que toda la emisión de datos
9         // la controla el servicio singleton 'SerialService' utilizando
10        IHubContext<SerialHub>.
11        // Sin embargo, permite que los clientes se conecten mediante WebSockets
12        directamente.
13
14        public override async Task OnConnectedAsync()
15        {
16            // Este método se dispara automáticamente cuando un navegador web se
17            // conecta.
18            // Es útil si quisieramos registrar información del cliente
19            // conectado en el futuro.
20            await base.OnConnectedAsync();
21        }
22    }
23 }

```

7.2.1 ¿Qué significan los componentes de este código?

- `using Microsoft.AspNetCore.SignalR;`: Importa la librería oficial de SignalR que provee la clase Hub.
- `namespace LedControllerService.Hubs;`: Define que la clase se encuentra en la carpeta lógica Hubs de tu proyecto.
- `public class SerialHub : Hub`: La palabra clave `:` significa **herencia**. Le estamos indicando a C# que nuestra clase SerialHub hereda todas las propiedades y comportamientos de la clase base Hub de Microsoft, convirtiéndose oficialmente en un canal de WebSockets de SignalR.
- `public override async Task OnConnectedAsync()`:
 - `override` (Sobrescribir): Indica que estamos reemplazando el comportamiento por defecto de lo que hace SignalR cuando alguien se conecta.
 - `async Task`: Define que es un método asíncrono (que no congela la ejecución mientras espera una respuesta de red).

Advertencias después de guardar el archivo SerialHub.cs

Al guardar este archivo, la advertencia de `using LedControllerService.Hubs;` en el archivo `SerialService.cs` desaparecerán.

7.3 Crear el controlador API (SerialController.cs)

El controlador es la puerta de entrada para las peticiones HTTP convencionales (REST) del frontend. Mientras que SignalR maneja la recepción continua de datos en tiempo real, el controlador maneja las órdenes directas que realiza el usuario mediante clics en la interfaz (como "conéctame a este puerto", "desconéctame" o "envía este comando al LED").

INYECCIÓN DE DEPENDENCIAS El controlador no tiene idea de cómo hablar con el puerto serie. En su lugar, el constructor recibe nuestro SerialService y le delega todo el trabajo.

EXPONENTS EXPUESTOS listado de endpoints

1. GET /api/serial/ports: Obtiene la lista de puertos COM
 2. GET /api/serial/status: Obtiene el estado actual de la conexión.
 3. POST /api/serial/connect: Ordena la conexión a un puerto y baudios específicos.
 4. POST /api/serial/disconnect: Cierra la conexión activa.
 5. POST /api/serial/send:Envía un comando a la tarjeta (0, 1, ?).
1. En tu editor, ve a la carpeta que ya existe en tu proyecto: Service/LedController Service/Controllers/
 2. Dentro de esa carpeta, crea un archivo llamado SerialController.cs.
 3. Pega el siguiente código en el archivo:

```

1 using LedControllerService.Services;
2 using Microsoft.AspNetCore.Mvc;
3
4 namespace LedControllerService.Controllers
5 {
6     [ApiController]
7     [Route("api/[controller]")] // Define la ruta base como: api/serial
8     public class SerialController : ControllerBase
9     {
10         private readonly SerialService _serialService;
11
12         // El constructor recibe el servicio singleton SerialService por
13         // Inyección de Dependencias
14         public SerialController(SerialService serialService)
15         {
16             _serialService = serialService;
17         }
18
19         // GET: api/serial/ports
20         [HttpGet("ports")]
21         public ActionResult<string[]> GetAvailablePorts()
22         {
23             return Ok(_serialService.GetAvailablePorts());
24         }
25
26         // GET: api/serial/status
27         [HttpGet("status")]
28         public ActionResult GetStatus()
29         {
30             return Ok(_serialService.GetStatus());
31         }
32     }

```

```

32 // POST: api/serial/connect
33 [HttpPost("connect")]
34 public ActionResult Connect([FromBody] ConnectRequest request)
35 {
36     if (string.IsNullOrEmpty(request.PortName))
37     {
38         return BadRequest("El nombre del puerto es obligatorio.");
39     }
40
41     if (request.BaudRate <= 0)
42     {
43         return BadRequest("La velocidad de baudios debe ser mayor a cero.
44     ");
45     }
46
47     string error;
48     bool success = _serialService.Connect(request.PortName, request.
49 BaudRate, out error);
50
51     if (success)
52     {
53         return Ok(new { Message = "Conectado exitosamente.", Success =
54 true });
55     }
56     else
57     {
58         return StatusCode(500, new { Message = $"Error de conexion: {
59 error}", Success = false });
60     }
61 }
62
63 // POST: api/serial/disconnect
64 [HttpPost("disconnect")]
65 public ActionResult Disconnect()
66 {
67     _serialService.Disconnect();
68     return Ok(new { Message = "Desconectado exitosamente.", Success =
69 true });
70 }
71
72 // POST: api/serial/send
73 [HttpPost("send")]
74 public ActionResult SendCommand([FromBody] SendCommandRequest request)
75 {
76     if (string.IsNullOrEmpty(request.Command))
77     {
78         return BadRequest("El comando no puede estar vacio.");
79     }
80
81     bool success = _serialService.SendCommand(request.Command);
82
83     if (success)
84     {
85         return Ok(new { Message = "Comando enviado.", Success = true });
86     }
87     else
88     {
89         return BadRequest(new { Message = "No se pudo enviar el comando.
90 El puerto esta conectado?", Success = false });
91     }
92 }

```

```

87     }
88
89     // Modelos de datos para recibir los parametros del cuerpo de las peticiones
90     // JSON (POST)
91     public class ConnectRequest
92     {
93         public string PortName { get; set; } = string.Empty;
94         public int BaudRate { get; set; } = 115200;
95     }
96
97     public class SendCommandRequest
98     {
99         public string Command { get; set; } = string.Empty;
100     }

```

Detallles del código

- [ApiController]: Le dice a .NET que esta clase responderá peticiones de red HTTP y aplicará validaciones automáticas a los modelos de datos de entrada.
- [Route("api/[controller]")]: Configura la URL. La palabra [controller] se reemplaza automáticamente por el nombre de la clase sin la palabra "Controller", por lo que todas las peticiones a esta API comenzarán con /api/serial.
- [FromBody]: Indica al compilador que el contenido de la petición (el JSON con el puerto o comando) viene en el cuerpo de la solicitud HTTP (Body) y debe ser transformado automáticamente a las clases ConnectRequest o SendCommandRequest.

7.4 Configuración del archivo Program.cs

El archivo Program.cs es el punto de entrada principal del servidor. Cuando inicias tu API con dotnet run, es este archivo el que se ejecuta primero.

Para que todo nuestro sistema funcione, debemos configurarle los siguientes servicios básicos al constructor del servidor:

CONFIGURAR LA POLÍTICA DE CORS: Por seguridad, los navegadores bloquean llamadas entre diferentes puertos (Angular corre en el puerto 4200 y C# corre en el puerto 5200). Aquí le daremos permiso explícito a `http://localhost:4200` para realizar peticiones y conectarse por WebSockets (usando `.AllowCredentials()`)

INTECTAR EL SerialService: Registraremos este servicio para que esté siempre activo y disponible para el controlador.

ACTIVAR SIGNALR : Habilita el motor de WebSockets en el backend.

MAPEAR EL HUB DE WEBSOCKETS: Configura la dirección URL física (/hubs/serial) que escuchará las conexiones de SignalR.

FORZAR EL PUERTO DEL SERVIDOR: Obligaremos a la aplicación a correr exactamente en `http://localhost:5200` para que coincida con la configuración de llamadas de Angular.

1. En el editor, abre el archivo existente Service/LedControllerService/Program.cs
2. Borra todo su contenido original y reemplázalo por el siguiente código estructurado:

```

1 using LedControllerService.Hubs;
2 using LedControllerService.Services;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 // 1. Configurar CORS (Cross-Origin Resource Sharing) para Angular
7 builder.Services.AddCors(options =>
8 {
9     options.AddPolicy("CorsPolicy", policy =>
10     {
11         policy.WithOrigins("http://localhost:4200") // Permite el origen del
12             frontend
13             .AllowAnyHeader()
14             .AllowAnyMethod()
15             .AllowCredentials(); // Obligatorio para permitir conexiones de
16             WebSockets de SignalR
17     });
18 });
19
20 // 2. Agregar el soporte para los controladores API REST tradicionales
21 builder.Services.AddControllers();
22
23 // 3. Agregar el soporte para SignalR (WebSockets)
24 builder.Services.AddSignalR();
25
26 // 4. Registrar nuestro SerialService como Singleton (instancia unica)
27 builder.Services.AddSingleton<SerialService>();
28
29 // 5. Agregar OpenAPI (Swagger) para documentacion y pruebas
30 builder.Services.AddOpenApi();
31
32 var app = builder.Build();
33
34 // Configurar el pipeline de peticiones HTTP
35 if (app.Environment.IsDevelopment())
36 {
37     app.MapOpenApi();
38 }
39
40 // 6. Activar la politica de CORS antes del ruteo de controladores y hubs
41 app.UseCors("CorsPolicy");
42
43 app.UseAuthorization();
44
45 // 7. Mapear los Controladores de la API REST
46 app.MapControllers();
47
48 // 8. Mapear la ruta URL para el Hub de SignalR
49 app.MapHub<SerialHub>("/hubs/serial");
50
51 // 9. Forzar al servidor a escuchar en el puerto local 5200
52 app.Run("http://localhost:5200");

```

- `builder.Services.AddSingleton<SerialService>();`: Le dice a .NET: "Crea una única instancia de `SerialService` al arrancar. Cuando un controlador (`Serial Controller`) la pida en su constructor, dale siempre esta misma instancia" (evita que se intente abrir el puerto USB varias veces).

- `app.UseCors("CorsPolicy");`: Activa el semáforo de seguridad de red en el servidor. Debe ir colocado antes de mapear controladores o hubs para que las peticiones web no sean rechazadas.
- `app.MapHub<SerialHub>("/hubs/serial");`: Mapea la dirección de WebSockets. El cliente de Angular intentará conectarse a `http://localhost:5200/hubs/serial`.

Con este paso, el código del backend está 100% completo. Para confirmar que no hay errores de sintaxis y que todo está listo para funcionar, abre una terminal de VS Code y realiza lo siguiente:

1. Navega a la carpeta del backend:

```
cd "C:\VisualStudioProjects\NucleoBoard_LedControl\Service\LedControllerService"
```

2. Ejecuta la compilación de prueba:

```
dotnet build
```

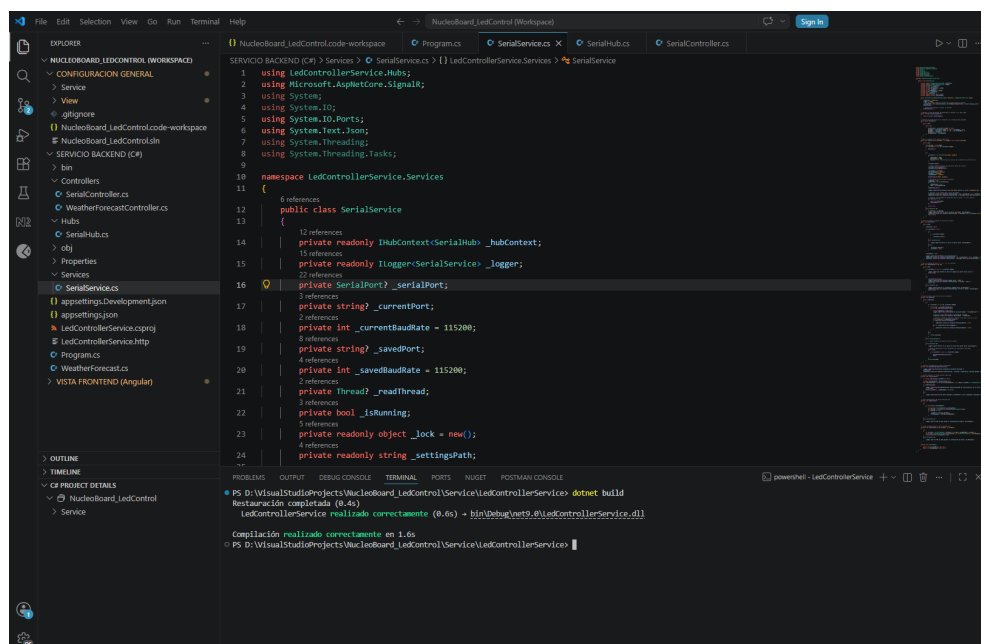


Figura 26: Compilación del backend.